INDEX

No.	Program Title	Page
1	Demonstration of Storage Classes in C	2
2	Menu-Driven Array Operations Using Global Variables	4
3	Menu-Driven Array Operations Using Local Variables	9
4	Search All Occurrences of an Element	14
5	Sort an Array in Ascending Order	16
6	Display an Array Using Recursion	18
7	Display an Array in Reverse Using Recursion	20
8	Matrix Operations	22
9	Reverse a String in the Same Array	27
10	Sort n Strings in Ascending Order	28
11	Stack Operations Using an Array	31
12	Evaluate a Push and Pop Sequence	33
13	Reverse a String Using a Stack	34
14	Infix to Postfix Conversion	35
15	Infix to Prefix Conversion	39
16	Evaluate an Infix Expression	44
17	Read and Display a Polynomial Using an Array	49
18	Add Two Polynomials Using Arrays	52
19	Subtract Two Polynomials Using Arrays	55
20	Multiply Two Polynomials Using Arrays	58
21	Date Structure	61
22	Employee Structure	63
23	Read and Display a Polynomial Using Structures	67
24	Add Two Polynomials Using Structures	70
25	Subtract Two Polynomials Using Structures	75
26	Multiply Two Polynomials Using Structures	80
27	Reverse a String	85
28	Sort n Strings in Ascending Order	87
29	Queue Using an Array	89
30	Circular Queue Using an Array	91
31	Dynamic Memory Allocation Functions	93
32	Mean Using malloc	95
33	Mode Using calloc	97
34	Array of Books Using Pointers	99
35	Variable-Length String Using realloc	101
36	Read and Display a Sparse Matrix	103
37	Add Two Sparse Matrices	106
38	Multiply Two Sparse Matrices	110
39	Heapsort	114
40	Priority Queue Using an Array	116
41	Linked List Creation and Display	118
42	Insert a Node in a Linked List	120
43	Delete a Node from a Linked List	123
44	Display a Linked List in Reverse Order	127
45	Sort a Linked List	130
46	Polynomial Using a Linked List	133
47	Linked List of Names	138
48	Linked Stack, Queue, and Linked Lists	142
49	Linear Search	156
50	Binary Search for Integers	158
51	Binary Search for Strings	160
52	Binary Search Tree Traversals	162
53	Inorder Traversal Without Recursion	165
54	Preorder Traversal Without Recursion	167
55	Postorder Traversal Without Recursion	170
56	Binary Search Tree Using Names	173
Project	Student Record Management System	175

Program No: 1 | **Date:** 10-07-2026

Program Title: Demonstration of Storage Classes in C

Description:

Demonstrates automatic (local), external (global), static, and register storage classes in C.

Author: Abhishek Vincent Roll No: 03 Date: 10-07-2026

Source Code:

```
#include <stdio.h>

extern int global_variable;
int global_variable = 10;

void show_local_variable(void)
{
    auto int local_variable = 5;

    printf("Local variable: %d\n", local_variable);
}

void show_global_variable(void)
{
    printf("Global variable accessed through extern declaration: %d\n",
           global_variable);
}

void show_static_variable(void)
{
    static int static_variable = 1;

    printf("Static variable: %d\n", static_variable);
    static_variable++;
}

void show_register_variable(void)
{
    register int register_variable;

    for (register_variable = 1; register_variable <= 3; register_variable++)
    {
        printf("Register variable: %d\n", register_variable);
    }
}

int main(void)
{
    printf("Storage class demonstration\n\n");

    show_local_variable();
    show_global_variable();

    printf("First call to the static variable function:\n");
    show_static_variable();
    printf("Second call to the static variable function:\n");
    show_static_variable();

    show_register_variable();
```

```
    return 0;
}
```

Output Text:

```
Storage class demonstration

Local variable: 5
Global variable accessed through extern declaration: 10
First call to the static variable function:
Static variable: 1
Second call to the static variable function:
Static variable: 2
Register variable: 1
Register variable: 2
Register variable: 3
```

Program No: 2 | **Date:** 14-07-2026

Program Title: Menu-Driven Array Operations Using Global Variables

Description:

Implements a menu-driven program to insert, search, delete, and sort elements in an array using global variables.

Author: Abhishek Vincent Roll No: 03 Date: 14-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

int array[MAX];
int size = 0;

void display()
{
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("Array: ");
    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }
    printf("\n");
}

void insert_element()
{
    int value;
    int position;
    int i;

    if (size == MAX)
    {
        printf("Array is full.\n");
        return;
    }

    printf("Enter the value to insert: ");
    scanf("%d", &value);
    printf("Enter the position (1 to %d): ", size + 1);
    scanf("%d", &position);

    if (position < 1 || position > size + 1)
    {
        printf("Invalid position.\n");
        return;
    }

    for (i = size; i >= position; i--)
```

```
    {
        array[i] = array[i - 1];
    }

    array[position - 1] = value;
    size++;
    printf("Element inserted.\n");
}

void search_element()
{
    int value;
    int i;

    printf("Enter the value to search: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            printf("Element found at position %d.\n", i + 1);
            return;
        }
    }

    printf("Element not found.\n");
}

void delete_element()
{
    int value;
    int position = -1;
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("Enter the value to delete: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Element not found.\n");
        return;
    }

    for (i = position; i < size - 1; i++)
    {
        array[i] = array[i + 1];
    }
```

```
    size--;
    printf("First occurrence deleted.\n");
}

void sort_elements()
{
    int i;
    int j;
    int temporary;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                temporary = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temporary;
            }
        }
    }

    printf("Array sorted in ascending order.\n");
}

int main(void)
{
    int choice;

    do
    {
        printf("\n1. Insert\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Delete\n");
        printf("5. Sort\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                insert_element();
                break;
            case 2:
                display();
                break;
            case 3:
                search_element();
                break;
            case 4:
                delete_element();
                break;
            case 5:
                sort_elements();
                break;
            case 6:
                printf("Program ended.\n");
                break;
            default:
```

```
                printf("Invalid choice.\n");
        }
    } while (choice != 6);

    return 0;
}
```

Output Text:

```
1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 10
Enter the position (1 to 1): 1
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 20
Enter the position (1 to 2): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 30
Enter the position (1 to 3): 3
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 25
Enter the position (1 to 4): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 20 30
```

```
1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 3
Enter the value to search: 20
Element found at position 3.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 4
Enter the value to delete: 20
First occurrence deleted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 30

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 5
Array sorted in ascending order.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 30

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 6
Program ended.
```

Program No: 3 | **Date:** 20-07-2026

Program Title: Menu-Driven Array Operations Using Local Variables

Description:

Implements a menu-driven program to insert, search, delete, and sort elements in an array using local variables passed to functions.

Author: Abhishek Vincent Roll No: 03 Date: 20-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void display(int array[], int size)
{
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("Array: ");
    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }
    printf("\n");
}

int insert_element(int array[], int size)
{
    int value;
    int position;
    int i;

    if (size == MAX)
    {
        printf("Array is full.\n");
        return size;
    }

    printf("Enter the value to insert: ");
    scanf("%d", &value);
    printf("Enter the position (1 to %d): ", size + 1);
    scanf("%d", &position);

    if (position < 1 || position > size + 1)
    {
        printf("Invalid position.\n");
        return size;
    }

    for (i = size; i >= position; i--)
    {
        array[i] = array[i - 1];
    }
```

```
    array[position - 1] = value;
    printf("Element inserted.\n");
    return size + 1;
}

void search_element(int array[], int size)
{
    int value;
    int i;

    printf("Enter the value to search: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            printf("Element found at position %d.\n", i + 1);
            return;
        }
    }

    printf("Element not found.\n");
}

int delete_element(int array[], int size)
{
    int value;
    int position = -1;
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
        return size;
    }

    printf("Enter the value to delete: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Element not found.\n");
        return size;
    }

    for (i = position; i < size - 1; i++)
    {
        array[i] = array[i + 1];
    }

    printf("First occurrence deleted.\n");
    return size - 1;
```

```
}

void sort_elements(int array[], int size)
{
    int i;
    int j;
    int temporary;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                temporary = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temporary;
            }
        }
    }

    printf("Array sorted in ascending order.\n");
}

int main(void)
{
    int array[MAX];
    int size = 0;
    int choice;

    do
    {
        printf("\n1. Insert\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Delete\n");
        printf("5. Sort\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                size = insert_element(array, size);
                break;
            case 2:
                display(array, size);
                break;
            case 3:
                search_element(array, size);
                break;
            case 4:
                size = delete_element(array, size);
                break;
            case 5:
                sort_elements(array, size);
                break;
            case 6:
                printf("Program ended.\n");
                break;
            default:
                printf("Invalid choice.\n");
```

```
        }
    } while (choice != 6);

    return 0;
}
```

Output Text:

```
1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 10
Enter the position (1 to 1): 1
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 20
Enter the position (1 to 2): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 30
Enter the position (1 to 3): 3
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 25
Enter the position (1 to 4): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 20 30

1. Insert
```

```
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 3
Enter the value to search: 20
Element found at position 3.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 4
Enter the value to delete: 20
First occurrence deleted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 30

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 5
Array sorted in ascending order.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 10 25 30

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 6
Program ended.
```

Program No: 4 | **Date:** 22-07-2026

Program Title: Search All Occurrences of an Element

Description:

Finds and displays all positions of a given element in an integer array.

Author: Abhishek Vincent Roll No: 03 Date: 22-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }
}

void display_occurrences(int array[], int size, int value)
{
    int i;
    int found = 0;

    printf("Positions of %d: ", value);
    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            printf("%d ", i + 1);
            found = 1;
        }
    }

    if (found == 0)
    {
        printf("Element not found");
    }
    printf("\n");
}

int main(void)
{
    int array[MAX];
    int size;
    int value;

    printf("Enter the size of the array (maximum %d): ", MAX);
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
```

```
    }

    read_array(array, size);
    printf("Enter the element to search: ");
    scanf("%d", &value);
    display_occurrences(array, size, value);

    return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 10
Enter element 4: 30
Enter element 5: 10
Enter the element to search: 10
Positions of 10: 1 3 5
```

Program No: 5 | **Date:** 22-07-2026

Program Title: Sort an Array in Ascending Order

Description:

Reads, displays, and sorts an integer array in ascending order using separate functions.

Author: Abhishek Vincent Roll No: 03 Date: 22-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

int array[MAX];
int size = 0;

void read_element(void)
{
    printf("Enter element %d: ", size + 1);
    scanf("%d", &array[size]);
    size++;
}

void display_array(void)
{
    int i;

    printf("Array: ");

    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }

    printf("\n");
}

void sort_array(void)
{
    int i;
    int j;
    int temporary;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                temporary = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temporary;
            }
        }
    }
}

int main(void)
{
```

```
    read_element();
    read_element();
    read_element();
    read_element();

    printf("Before sorting:\n");
    display_array();

    sort_array();

    printf("After sorting:\n");
    display_array();

    return 0;
}
```

Output Text:

```
Enter element 1: 40
Enter element 2: 10
Enter element 3: 30
Enter element 4: 20
Before sorting:
Array: 40 10 30 20
After sorting:
Array: 10 20 30 40
```

Program No: 6 | **Date:** 02-08-2026

Program Title: Display an Array Using Recursion

Description:

Prints the elements of an integer array in their original order using recursion.

Author: Abhishek Vincent Roll No: 03 Date: 02-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }
}

void display_in_same_order(int array[], int size, int position)
{
    if (position == size)
    {
        return;
    }

    printf("%d ", array[position]);
    display_in_same_order(array, size, position + 1);
}

int main()
{
    int array[MAX];
    int size;

    printf("Enter the size of the array (maximum %d): ", MAX);
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
    }

    read_array(array, size);

    printf("Array in the same order: ");
    display_in_same_order(array, size, 0);
    printf("\n");

    return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 23
Enter element 2: 51
Enter element 3: 2
Enter element 4: 67
Enter element 5: 45
Array in the same order: 23 51 2 67 45
```

Program No: 7 | **Date:** 02-08-2026

Program Title: Display an Array in Reverse Using Recursion

Description:

Prints the elements of an integer array in reverse order using recursion.

Author: Abhishek Vincent Roll No: 03 Date: 02-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }
}

void display_in_reverse(int array[], int position)
{
    if (position < 0)
    {
        return;
    }

    printf("%d ", array[position]);
    display_in_reverse(array, position - 1);
}

int main()
{
    int array[MAX];
    int size;

    printf("Enter the size of the array (maximum %d): ", MAX);
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
    }

    read_array(array, size);

    printf("Array in reverse order: ");
    display_in_reverse(array, size - 1);
    printf("\n");

    return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 23
Enter element 2: 51
Enter element 3: 2
Enter element 4: 67
Enter element 5: 45
Array in reverse order: 45 67 2 51 23
```

Program No: 8 | **Date:** 28-07-2026

Program Title: Matrix Operations

Description:

Performs addition, subtraction, multiplication, transpose, and determinant operations on matrices.

Author: Abhishek Vincent Roll No: 03 Date: 28-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 3

void read_matrix(int matrix[][MAX], int rows, int columns)
{
    int i;
    int j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            printf("Enter element at row %d, column %d: ", i + 1, j + 1);
            scanf("%d", &matrix[i][j]);
        }
    }
}

void display_matrix(int matrix[][MAX], int rows, int columns)
{
    int i;
    int j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

void add_matrices()
{
    int first[MAX][MAX];
    int second[MAX][MAX];
    int result[MAX][MAX];
    int rows;
    int columns;
    int i;
    int j;

    printf("Enter rows and columns (maximum %d): ", MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
```

```
    }

    printf("Enter the first matrix:\n");
    read_matrix(first, rows, columns);
    printf("Enter the second matrix:\n");
    read_matrix(second, rows, columns);

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            result[i][j] = first[i][j] + second[i][j];
        }
    }

    printf("Sum of the matrices:\n");
    display_matrix(result, rows, columns);
}

void subtract_matrices(void)
{
    int first[MAX][MAX];
    int second[MAX][MAX];
    int result[MAX][MAX];
    int rows;
    int columns;
    int i;
    int j;

    printf("Enter rows and columns (maximum %d): ", MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    printf("Enter the first matrix:\n");
    read_matrix(first, rows, columns);
    printf("Enter the second matrix:\n");
    read_matrix(second, rows, columns);

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            result[i][j] = first[i][j] - second[i][j];
        }
    }

    printf("First matrix minus second matrix:\n");
    display_matrix(result, rows, columns);
}

void multiply_matrices(void)
{
    int first[MAX][MAX];
    int second[MAX][MAX];
    int result[MAX][MAX];
    int first_rows;
    int first_columns;
    int second_rows;
    int second_columns;
```

```
    int i;
    int j;
    int k;

    printf("Enter rows and columns of the first matrix: ");
    scanf("%d %d", &first_rows, &first_columns);
    printf("Enter rows and columns of the second matrix: ");
    scanf("%d %d", &second_rows, &second_columns);

    if (first_rows < 1 || first_rows > MAX || first_columns < 1 ||
        first_columns > MAX || second_rows < 1 || second_rows > MAX ||
        second_columns < 1 || second_columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    if (first_columns != second_rows)
    {
        printf("Matrix multiplication is not possible.\n");
        return;
    }

    printf("Enter the first matrix:\n");
    read_matrix(first, first_rows, first_columns);
    printf("Enter the second matrix:\n");
    read_matrix(second, second_rows, second_columns);

    for (i = 0; i < first_rows; i++)
    {
        for (j = 0; j < second_columns; j++)
        {
            result[i][j] = 0;
            for (k = 0; k < first_columns; k++)
            {
                result[i][j] = result[i][j] +
                               first[i][k] * second[k][j];
            }
        }
    }

    printf("Product of the matrices:\n");
    display_matrix(result, first_rows, second_columns);
}

void transpose_matrix(void)
{
    int matrix[MAX][MAX];
    int rows;
    int columns;
    int i;
    int j;

    printf("Enter rows and columns (maximum %d): ", MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    read_matrix(matrix, rows, columns);
```

```
    printf("Transpose of the matrix:\n");
    for (i = 0; i < columns; i++)
    {
        for (j = 0; j < rows; j++)
        {
            printf("%d ", matrix[j][i]);
        }
        printf("\n");
    }
}

void find_determinant(void)
{
    int matrix[MAX][MAX];
    int order;
    int determinant;

    printf("Enter the order of the matrix (2 or 3): ");
    scanf("%d", &order);
    if (order != 2 && order != 3)
    {
        printf("Only 2x2 and 3x3 matrices are supported.\n");
        return;
    }

    read_matrix(matrix, order, order);

    if (order == 2)
    {
        determinant = matrix[0][0] * matrix[1][1] -
                      matrix[0][1] * matrix[1][0];
    }
    else
    {
        determinant = matrix[0][0] *
                        (matrix[1][1] * matrix[2][2] -
                         matrix[1][2] * matrix[2][1]) -
                      matrix[0][1] *
                        (matrix[1][0] * matrix[2][2] -
                         matrix[1][2] * matrix[2][0]) +
                      matrix[0][2] *
                        (matrix[1][0] * matrix[2][1] -
                         matrix[1][1] * matrix[2][0]);
    }

    printf("Determinant = %d\n", determinant);
}

int main(void)
{
    add_matrices();
    subtract_matrices();
    multiply_matrices();
    transpose_matrix();
    find_determinant();

    return 0;
}
```

Output Text:

```
Enter rows and columns (maximum 3): 2 2
Enter the first matrix:
```

```
Enter element at row 1, column 1: 1
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 4
Enter the second matrix:
Enter element at row 1, column 1: 5
Enter element at row 1, column 2: 6
Enter element at row 2, column 1: 7
Enter element at row 2, column 2: 8
Sum of the matrices:
6 8
10 12
Enter rows and columns (maximum 3): 2 2
Enter the first matrix:
Enter element at row 1, column 1: 5
Enter element at row 1, column 2: 6
Enter element at row 2, column 1: 7
Enter element at row 2, column 2: 8
Enter the second matrix:
Enter element at row 1, column 1: 1
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 4
First matrix minus second matrix:
4 4
4 4
Enter rows and columns of the first matrix: 2 2
Enter rows and columns of the second matrix: 2 2
Enter the first matrix:
Enter element at row 1, column 1: 1
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 4
Enter the second matrix:
Enter element at row 1, column 1: 5
Enter element at row 1, column 2: 6
Enter element at row 2, column 1: 7
Enter element at row 2, column 2: 8
Product of the matrices:
19 22
43 50
Enter rows and columns (maximum 3): 2 2
Enter element at row 1, column 1: 1
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 4
Transpose of the matrix:
1 3
2 4
Enter the order of the matrix (2 or 3): 2
Enter element at row 1, column 1: 1
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 4
Determinant = -2
```

Program No: 9 | **Date:** 24-07-2026

Program Title: Reverse a String in the Same Array

Description:

Reads a string and reverses it in the same array.

Author: Abhishek Vincent Roll No: 03 Date: 24-07-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

void read_string(char text[])
{
    printf("Enter a string: ");
    scanf(" %99[^\n]", text);
}

void reverse_string(char text[])
{
    int length = 0;
    int left;
    int right;
    char temporary;

    while (text[length] != '\0')
    {
        length++;
    }

    left = 0;
    right = length - 1;
    while (left < right)
    {
        temporary = text[left];
        text[left] = text[right];
        text[right] = temporary;
        left++;
        right--;
    }
}

int main()
{
    char text[MAX_LENGTH];

    read_string(text);
    reverse_string(text);
    printf("String in reverse order: %s\n", text);

    return 0;
}
```

Output Text:

```
Enter a string: hello world
String in reverse order: dlrow olleh
```

Program No: 10 | **Date:** 24-07-2026

Program Title: Sort n Strings in Ascending Order

Description:

Reads n strings and displays them in ascending order.

Author: Abhishek Vincent Roll No: 03 Date: 24-07-2026

Source Code:

```
#include <stdio.h>

#define MAX_STRINGS 20
#define MAX_LENGTH 50

int read_strings(char strings[][MAX_LENGTH])
{
    int count;
    int i;

    printf("Enter the number of strings (maximum %d): ", MAX_STRINGS);
    scanf("%d", &count);

    if (count < 1 || count > MAX_STRINGS)
    {
        return 0;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter string %d: ", i + 1);
        scanf(" %49[^\n]", strings[i]);
    }

    return count;
}

int compare_strings(char first[], char second[])
{
    int i = 0;

    while (first[i] != '\0' && second[i] != '\0')
    {
        if (first[i] < second[i])
        {
            return -1;
        }
        if (first[i] > second[i])
        {
            return 1;
        }
        i++;
    }

    if (first[i] == '\0' && second[i] == '\0')
    {
        return 0;
    }
    if (first[i] == '\0')
    {
```

```
        return -1;
    }
    return 1;
}

void sort_strings(char strings[][MAX_LENGTH], int count)
{
    int i;
    int j;
    int k;
    char temporary;

    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - i - 1; j++)
        {
            if (compare_strings(strings[j], strings[j + 1]) > 0)
            {
                for (k = 0; k < MAX_LENGTH; k++)
                {
                    temporary = strings[j][k];
                    strings[j][k] = strings[j + 1][k];
                    strings[j + 1][k] = temporary;
                }
            }
        }
    }
}

void display_strings(char strings[][MAX_LENGTH], int count)
{
    int i;

    for (i = 0; i < count; i++)
    {
        printf("%s\n", strings[i]);
    }
}

int main()
{
    char strings[MAX_STRINGS][MAX_LENGTH];
    int count;

    count = read_strings(strings);
    if (count == 0)
    {
        printf("Invalid number of strings.\n");
        return 0;
    }

    sort_strings(strings, count);

    printf("Strings in ascending order:\n");
    display_strings(strings, count);

    return 0;
}
```

Output Text:

```
Enter the number of strings (maximum 20): 4
Enter string 1: banana
```

```
Enter string 2: apple
Enter string 3: cherry
Enter string 4: date
Strings in ascending order:
apple
banana
cherry
date
```

Program No: 11 | **Date:** 05-08-2026

Program Title: Stack Operations Using an Array

Description:

Implements push, pop, peek, and display operations on a stack using an array.

Author: Abhishek Vincent Roll No: 03 Date: 05-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push()
{
    int value;

    if (top == MAX - 1)
    {
        printf("Stack overflow.\n");
        return;
    }

    printf("Enter the value to push: ");
    scanf("%d", &value);
    top++;
    stack[top] = value;
    printf("Value pushed.\n");
}

void pop()
{
    if (top == -1)
    {
        printf("Stack underflow.\n");
        return;
    }

    printf("Popped value: %d\n", stack[top]);
    top--;
}

void peek()
{
    if (top == -1)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Top value: %d\n", stack[top]);
}

void display()
{
    int i;
```

```
    if (top == -1)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Stack from top to bottom: ");
    for (i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

int main(void)
{
    push();
    push();
    push();

    display();

    peek();

    pop();

    display();

    return 0;
}
```

Output Text:

```
Enter the value to push: 10
Value pushed.
Enter the value to push: 20
Value pushed.
Enter the value to push: 30
Value pushed.
Stack from top to bottom: 30 20 10
Top value: 30
Popped value: 30
Stack from top to bottom: 20 10
```

Program No: 12 | **Date:** 05-08-2026

Program Title: Evaluate a Push and Pop Sequence

Description:

Evaluates the given push and pop sequence on a LIFO stack and prints the values returned by the pop operations.

Author: Abhishek Vincent Roll No: 03 Date: 05-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 30

char stack[MAX];
int top = -1;

void push(char value)
{
    top++;
    stack[top] = value;
}

char pop()
{
    char value = stack[top];

    top--;
    return value;
}

int main()
{
    char operations[] = "EAS*Y*QUE***ST***IO*N***";
    int i = 0;

    printf("Values returned by pop operations: ");

    while (operations[i] != '\0')
    {
        if (operations[i] == '*')
        {
            printf("%c ", pop());
        }
        else
        {
            push(operations[i]);
        }
        i++;
    }

    printf("\n");
    return 0;
}
```

Output Text:

```
Values returned by pop operations: S Y E U Q T S A O N I E
```

Program No: 13 | **Date:** 07-08-2026

Program Title: Reverse a String Using a Stack

Description:

Reads a string and reverses it using a stack.

Author: Abhishek Vincent Roll No: 03 Date: 07-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

char stack[MAX_LENGTH];
int top = -1;

void push(char value)
{
    top++;
    stack[top] = value;
}

char pop(void)
{
    char value = stack[top];

    top--;
    return value;
}

int main(void)
{
    char text[MAX_LENGTH];
    int i = 0;

    printf("Enter a string: ");
    scanf(" %99[^\n]", text);

    while (text[i] != '\0')
    {
        push(text[i]);
        i++;
    }

    printf("String in reverse order: ");
    while (top != -1)
    {
        printf("%c", pop());
    }
    printf("\n");

    return 0;
}
```

Output Text:

```
Enter a string: data structures
String in reverse order: serutcurts atad
```

Program No: 14 | **Date:** 12-08-2026

Program Title: Infix to Postfix Conversion

Description:

Converts an infix expression to postfix using a stack.

Author: Abhishek Vincent Roll No: 03 Date: 12-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

char stack[MAX_LENGTH];
int top = -1;

void push(char value)
{
    top++;
    stack[top] = value;
}

char pop()
{
    char value = stack[top];

    top--;
    return value;
}

char peek()
{
    return stack[top];
}

int is_operand(char symbol)
{
    if (symbol >= 'A' && symbol <= 'Z')
    {
        return 1;
    }
    if (symbol >= 'a' && symbol <= 'z')
    {
        return 1;
    }
    if (symbol >= '0' && symbol <= '9')
    {
        return 1;
    }
    return 0;
}

int is_operator(char symbol)
{
    if (symbol == '+' || symbol == '-' || symbol == '*' ||
        symbol == '/' || symbol == '%' || symbol == '^')
    {
        return 1;
    }
```

```
    return 0;
}

int precedence(char symbol)
{
    if (symbol == '+' || symbol == '-')
    {
        return 1;
    }
    if (symbol == '*' || symbol == '/' || symbol == '%')
    {
        return 2;
    }
    if (symbol == '^')
    {
        return 3;
    }
    return 0;
}

int should_pop(char stack_symbol, char current_symbol)
{
    if (precedence(stack_symbol) > precedence(current_symbol))
    {
        return 1;
    }

    if (precedence(stack_symbol) == precedence(current_symbol) &&
        current_symbol != '^')
    {
        return 1;
    }

    return 0;
}

int main()
{
    char infix[MAX_LENGTH];
    char postfix[MAX_LENGTH];
    int i = 0;
    int output_position = 0;
    int expect_operand = 1;
    int valid = 1;
    char symbol;

    printf("Enter an infix expression without spaces: ");
    scanf("%99s", infix);

    while (infix[i] != '\0' && valid == 1)
    {
        symbol = infix[i];

        if (is_operand(symbol))
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else if (symbol >= '0' && symbol <= '9' &&
                     infix[i + 1] >= '0' && infix[i + 1] <= '9')
            {
                valid = 0;
```

```
            }
            else
            {
                postfix[output_position] = symbol;
                output_position++;
                expect_operand = 0;
                i++;
            }
        }
        else if (symbol == '(')
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else
            {
                push(symbol);
                i++;
            }
        }
        else if (symbol == ')')
        {
            if (expect_operand == 1)
            {
                valid = 0;
            }
            else
            {
                while (top != -1 && peek() != '(')
                {
                    postfix[output_position] = pop();
                    output_position++;
                }

                if (top == -1)
                {
                    valid = 0;
                }
                else
                {
                    pop();
                    expect_operand = 0;
                    i++;
                }
            }
        }
        else if (is_operator(symbol))
        {
            if (expect_operand == 1)
            {
                valid = 0;
            }
            else
            {
                while (top != -1 && peek() != '(' &&
                       should_pop(peek(), symbol) == 1)
                {
                    postfix[output_position] = pop();
                    output_position++;
                }

                push(symbol);
```

```
                expect_operand = 1;
                i++;
            }
        }
        else
        {
            valid = 0;
        }
    }

    if (expect_operand == 1)
    {
        valid = 0;
    }

    while (top != -1 && valid == 1)
    {
        if (peek() == '(')
        {
            valid = 0;
        }
        else
        {
            postfix[output_position] = pop();
            output_position++;
        }
    }

    if (valid == 1)
    {
        postfix[output_position] = '\0';
        printf("Postfix expression: %s\n", postfix);
    }
    else
    {
        printf("Invalid expression. Use single-character operands.\n");
    }

    return 0;
}
```

Output Text:

```
Enter an infix expression without spaces: A+B*C
Postfix expression: ABC*+
```

Program No: 15 | **Date:** 12-08-2026

Program Title: Infix to Prefix Conversion

Description:

Converts an infix expression to prefix using a stack.

Author: Abhishek Vincent Roll No: 03 Date: 12-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

char stack[MAX_LENGTH];
int top = -1;

void push(char value)
{
    top++;
    stack[top] = value;
}

char pop(void)
{
    char value = stack[top];

    top--;
    return value;
}

char peek(void)
{
    return stack[top];
}

int is_operand(char symbol)
{
    if (symbol >= 'A' && symbol <= 'Z')
    {
        return 1;
    }
    if (symbol >= 'a' && symbol <= 'z')
    {
        return 1;
    }
    if (symbol >= '0' && symbol <= '9')
    {
        return 1;
    }
    return 0;
}

int is_operator(char symbol)
{
    if (symbol == '+' || symbol == '-' || symbol == '*' ||
        symbol == '/' || symbol == '%' || symbol == '^')
    {
        return 1;
    }
```

```
    return 0;
}

int precedence(char symbol)
{
    if (symbol == '+' || symbol == '-')
    {
        return 1;
    }
    if (symbol == '*' || symbol == '/' || symbol == '%')
    {
        return 2;
    }
    if (symbol == '^')
    {
        return 3;
    }
    return 0;
}

int should_pop_for_prefix(char stack_symbol, char current_symbol)
{
    if (precedence(stack_symbol) > precedence(current_symbol))
    {
        return 1;
    }

    if (precedence(stack_symbol) == precedence(current_symbol) &&
        current_symbol == '^')
    {
        return 1;
    }

    return 0;
}

void reverse_and_swap(char source[], char destination[])
{
    int length = 0;
    int i;
    char symbol;

    while (source[length] != '\0')
    {
        length++;
    }

    for (i = 0; i < length; i++)
    {
        symbol = source[length - i - 1];
        if (symbol == '(')
        {
            destination[i] = ')';
        }
        else if (symbol == ')')
        {
            destination[i] = '(';
        }
        else
        {
            destination[i] = symbol;
        }
    }
```

```
    destination[length] = '\0';
}

int make_postfix(char infix[], char postfix[])
{
    int i = 0;
    int output_position = 0;
    int expect_operand = 1;
    int valid = 1;
    char symbol;

    top = -1;

    while (infix[i] != '\0' && valid == 1)
    {
        symbol = infix[i];

        if (is_operand(symbol))
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else if (symbol >= '0' && symbol <= '9' &&
                     infix[i + 1] >= '0' && infix[i + 1] <= '9')
            {
                valid = 0;
            }
            else
            {
                postfix[output_position] = symbol;
                output_position++;
                expect_operand = 0;
                i++;
            }
        }
        else if (symbol == '(')
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else
            {
                push(symbol);
                i++;
            }
        }
        else if (symbol == ')')
        {
            if (expect_operand == 1)
            {
                valid = 0;
            }
            else
            {
                while (top != -1 && peek() != '(')
                {
                    postfix[output_position] = pop();
                    output_position++;
                }

                if (top == -1)
```

```
                {
                    valid = 0;
                }
                else
                {
                    pop();
                    expect_operand = 0;
                    i++;
                }
            }
        }
        else if (is_operator(symbol))
        {
            if (expect_operand == 1)
            {
                valid = 0;
            }
            else
            {
                while (top != -1 && peek() != '(' &&
                       should_pop_for_prefix(peek(), symbol) == 1)
                {
                    postfix[output_position] = pop();
                    output_position++;
                }

                push(symbol);
                expect_operand = 1;
                i++;
            }
        }
        else
        {
            valid = 0;
        }
    }

    if (expect_operand == 1)
    {
        valid = 0;
    }

    while (top != -1 && valid == 1)
    {
        if (peek() == '(')
        {
            valid = 0;
        }
        else
        {
            postfix[output_position] = pop();
            output_position++;
        }
    }

    postfix[output_position] = '\0';
    return valid;
}

int main(void)
{
    char infix[MAX_LENGTH];
    char reversed[MAX_LENGTH];
```

```
    char postfix[MAX_LENGTH];
    char prefix[MAX_LENGTH];
    int length = 0;
    int i;
    int valid;

    printf("Enter an infix expression without spaces: ");
    scanf("%99s", infix);

    reverse_and_swap(infix, reversed);
    valid = make_postfix(reversed, postfix);

    if (valid == 1)
    {
        while (postfix[length] != '\0')
        {
            length++;
        }

        for (i = 0; i < length; i++)
        {
            prefix[i] = postfix[length - i - 1];
        }
        prefix[length] = '\0';

        printf("Prefix expression: %s\n", prefix);
    }
    else
    {
        printf("Invalid expression. Use single-character operands.\n");
    }

    return 0;
}
```

Output Text:

```
Enter an infix expression without spaces: A+B*C
Prefix expression: +A*BC
```

Program No: 16 | **Date:** 14-08-2026

Program Title: Evaluate an Infix Expression

Description:

Evaluates an infix expression using operand and operator stacks.

Author: Abhishek Vincent Roll No: 03 Date: 14-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

int value_stack[MAX_LENGTH];
char operator_stack[MAX_LENGTH];
int value_top = -1;
int operator_top = -1;
int error = 0;

void push_value(int value)
{
    value_top++;
    value_stack[value_top] = value;
}

int pop_value(void)
{
    int value = value_stack[value_top];

    value_top--;
    return value;
}

void push_operator(char symbol)
{
    operator_top++;
    operator_stack[operator_top] = symbol;
}

char pop_operator(void)
{
    char symbol = operator_stack[operator_top];

    operator_top--;
    return symbol;
}

char peek_operator(void)
{
    return operator_stack[operator_top];
}

int is_operator(char symbol)
{
    if (symbol == '+' || symbol == '-' || symbol == '*' ||
        symbol == '/' || symbol == '%' || symbol == '^')
    {
        return 1;
    }
```

```
    return 0;
}

int precedence(char symbol)
{
    if (symbol == '+' || symbol == '-')
    {
        return 1;
    }
    if (symbol == '*' || symbol == '/' || symbol == '%')
    {
        return 2;
    }
    if (symbol == '^')
    {
        return 3;
    }
    return 0;
}

int should_pop(char stack_symbol, char current_symbol)
{
    if (precedence(stack_symbol) > precedence(current_symbol))
    {
        return 1;
    }

    if (precedence(stack_symbol) == precedence(current_symbol) &&
        current_symbol != '^')
    {
        return 1;
    }

    return 0;
}

int power(int base, int exponent)
{
    int result = 1;
    int i;

    for (i = 0; i < exponent; i++)
    {
        result = result * base;
    }
    return result;
}

void apply_operator(void)
{
    int left;
    int right;
    int result;
    char symbol;

    if (value_top < 1 || operator_top == -1)
    {
        error = 1;
        return;
    }

    right = pop_value();
    left = pop_value();
```

```
    symbol = pop_operator();

    if (symbol == '+')
    {
        result = left + right;
    }
    else if (symbol == '-')
    {
        result = left - right;
    }
    else if (symbol == '*')
    {
        result = left * right;
    }
    else if (symbol == '^')
    {
        if (right < 0)
        {
            error = 1;
            return;
        }
        result = power(left, right);
    }
    else if (symbol == '/')
    {
        if (right == 0)
        {
            error = 1;
            return;
        }
        result = left / right;
    }
    else
    {
        if (right == 0)
        {
            error = 1;
            return;
        }
        result = left % right;
    }

    push_value(result);
}

int main(void)
{
    char expression[MAX_LENGTH];
    int i = 0;
    int number;
    int expect_operand = 1;
    int valid = 1;
    char symbol;

    printf("Enter an infix expression: ");
    scanf(" %99[^\n]", expression);

    while (expression[i] != '\0' && valid == 1 && error == 0)
    {
        symbol = expression[i];

        if (symbol == ' ')
        {
```

```
            i++;
        }
        else if (symbol >= '0' && symbol <= '9')
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else
            {
                number = 0;
                while (expression[i] >= '0' && expression[i] <= '9')
                {
                    number = number * 10 + expression[i] - '0';
                    i++;
                }
                push_value(number);
                expect_operand = 0;
            }
        }
        else if (symbol == '(')
        {
            if (expect_operand == 0)
            {
                valid = 0;
            }
            else
            {
                push_operator(symbol);
                i++;
            }
        }
        else if (symbol == ')')
        {
            if (expect_operand == 1)
            {
                valid = 0;
            }
            else
            {
                while (operator_top != -1 &&
                       peek_operator() != '(' && error == 0)
                {
                    apply_operator();
                }

                if (operator_top == -1)
                {
                    valid = 0;
                }
                else
                {
                    pop_operator();
                    expect_operand = 0;
                    i++;
                }
            }
        }
        else if (is_operator(symbol))
        {
            if (expect_operand == 1)
            {
                valid = 0;
```

```
            }
            else
            {
                while (operator_top != -1 &&
                       peek_operator() != '(' &&
                       should_pop(peek_operator(), symbol) == 1 &&
                       error == 0)
                {
                    apply_operator();
                }

                push_operator(symbol);
                expect_operand = 1;
                i++;
            }
        }
        else
        {
            valid = 0;
        }
    }

    if (expect_operand == 1)
    {
        valid = 0;
    }

    while (operator_top != -1 && valid == 1 && error == 0)
    {
        if (peek_operator() == '(')
        {
            valid = 0;
        }
        else
        {
            apply_operator();
        }
    }

    if (valid == 1 && error == 0 && value_top == 0)
    {
        printf("Result = %d\n", pop_value());
    }
    else if (error == 1)
    {
        printf("Cannot evaluate the expression. Check the operands or division by zero.\n")
    ;
    }
    else
    {
        printf("Invalid expression.\n");
    }

    return 0;
}
```

Output Text:

```
Enter an infix expression: 10+20*3
Result = 70
```

Program No: 17 | **Date:** 19-08-2026

Program Title: Read and Display a Polynomial Using an Array

Description:

Reads a polynomial using coefficient arrays and displays it.

Author: Abhishek Vincent Roll No: 03 Date: 19-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_DEGREE 20

int read_polynomial(int coefficients[])
{
    int degree;
    int i;

    printf("Enter the degree of the polynomial: ");
    scanf("%d", &degree);

    if (degree < 0 || degree > MAX_DEGREE)
    {
        printf("Invalid degree.\n");
        return -1;
    }

    for (i = degree; i >= 0; i--)
    {
        printf("Enter the coefficient of x^%d: ", i);
        scanf("%d", &coefficients[i]);
    }

    return degree;
}

void display_polynomial(int coefficients[], int degree)
{
    int i;
    int printed = 0;

    for (i = degree; i >= 0; i--)
    {
        if (coefficients[i] == 0)
        {
            continue;
        }

        if (printed == 1 && coefficients[i] > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficients[i] < 0)
        {
            printf(" - ");
        }
        else if (coefficients[i] < 0)
        {
            printf("-");
```

```
        }

        if (i == 0)
        {
            printf("%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
        }
        else if (i == 1)
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x");
            }
            else
            {
                printf("%dx", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
            }
        }
        else
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x^%d", i);
            }
            else
            {
                printf("%dx^%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i],
    i);
            }
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

int main(void)
{
    int coefficients[MAX_DEGREE + 1] = {0};
    int degree;

    degree = read_polynomial(coefficients);

    if (degree != -1)
    {
        printf("Polynomial: ");
        display_polynomial(coefficients, degree);
    }

    return 0;
}
```

Output Text:

```
Enter the degree of the polynomial: 3
Enter the coefficient of x^3: 2
Enter the coefficient of x^2: 0
Enter the coefficient of x^1: -3
```

```
Enter the coefficient of x^0: 4
Polynomial: 2x^3 - 3x + 4
```

Program No: 18 | **Date:** 19-08-2026

Program Title: Add Two Polynomials Using Arrays

Description:

Adds two polynomials represented using coefficient arrays.

Author: Abhishek Vincent Roll No: 03 Date: 19-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_DEGREE 20

int read_polynomial(int coefficients[])
{
    int degree;
    int i;

    printf("Enter the degree of the polynomial: ");
    scanf("%d", &degree);

    if (degree < 0 || degree > MAX_DEGREE)
    {
        printf("Invalid degree.\n");
        return -1;
    }

    for (i = degree; i >= 0; i--)
    {
        printf("Enter the coefficient of x^%d: ", i);
        scanf("%d", &coefficients[i]);
    }

    return degree;
}

void display_polynomial(int coefficients[], int degree)
{
    int i;
    int printed = 0;

    for (i = degree; i >= 0; i--)
    {
        if (coefficients[i] == 0)
        {
            continue;
        }

        if (printed == 1 && coefficients[i] > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficients[i] < 0)
        {
            printf(" - ");
        }
        else if (coefficients[i] < 0)
        {
            printf("-");
```

```
        }

        if (i == 0)
        {
            printf("%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
        }
        else if (i == 1)
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x");
            }
            else
            {
                printf("%dx", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
            }
        }
        else
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x^%d", i);
            }
            else
            {
                printf("%dx^%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i],
    i);
            }
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

int add_polynomials(int first[], int first_degree, int second[], int second_degree, int
    result[])
{
    int degree;
    int i;

    degree = first_degree > second_degree ? first_degree : second_degree;

    for (i = 0; i <= degree; i++)
    {
        result[i] = 0;

        if (i <= first_degree)
        {
            result[i] += first[i];
        }

        if (i <= second_degree)
        {
            result[i] += second[i];
        }
    }
```

```
    return degree;
}

int main(void)
{
    int first[MAX_DEGREE + 1] = {0};
    int second[MAX_DEGREE + 1] = {0};
    int result[MAX_DEGREE + 1] = {0};
    int first_degree;
    int second_degree;
    int result_degree;

    printf("First polynomial:\n");
    first_degree = read_polynomial(first);

    if (first_degree == -1)
    {
        return 0;
    }

    printf("Second polynomial:\n");
    second_degree = read_polynomial(second);

    if (second_degree == -1)
    {
        return 0;
    }

    result_degree = add_polynomials(first, first_degree, second, second_degree, result);

    printf("First polynomial: ");
    display_polynomial(first, first_degree);
    printf("Second polynomial: ");
    display_polynomial(second, second_degree);
    printf("Sum: ");
    display_polynomial(result, result_degree);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the degree of the polynomial: 2
Enter the coefficient of x^2: 3
Enter the coefficient of x^1: 2
Enter the coefficient of x^0: 1
Second polynomial:
Enter the degree of the polynomial: 2
Enter the coefficient of x^2: 1
Enter the coefficient of x^1: 0
Enter the coefficient of x^0: 4
First polynomial: 3x^2 + 2x + 1
Second polynomial: x^2 + 4
Sum: 4x^2 + 2x + 5
```

Program No: 19 | **Date:** 21-08-2026

Program Title: Subtract Two Polynomials Using Arrays

Description:

Subtracts one polynomial from another using coefficient arrays.

Author: Abhishek Vincent Roll No: 03 Date: 21-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_DEGREE 20

int read_polynomial(int coefficients[])
{
    int degree;
    int i;

    printf("Enter the degree of the polynomial: ");
    scanf("%d", &degree);

    if (degree < 0 || degree > MAX_DEGREE)
    {
        printf("Invalid degree.\n");
        return -1;
    }

    for (i = degree; i >= 0; i--)
    {
        printf("Enter the coefficient of x^%d: ", i);
        scanf("%d", &coefficients[i]);
    }

    return degree;
}

void display_polynomial(int coefficients[], int degree)
{
    int i;
    int printed = 0;

    for (i = degree; i >= 0; i--)
    {
        if (coefficients[i] == 0)
        {
            continue;
        }

        if (printed == 1 && coefficients[i] > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficients[i] < 0)
        {
            printf(" - ");
        }
        else if (coefficients[i] < 0)
        {
            printf("-");
```

```
        }

        if (i == 0)
        {
            printf("%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
        }
        else if (i == 1)
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x");
            }
            else
            {
                printf("%dx", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
            }
        }
        else
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x^%d", i);
            }
            else
            {
                printf("%dx^%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i],
    i);
            }
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

int subtract_polynomials(int first[], int first_degree, int second[], int second_degree,
    int result[])
{
    int degree;
    int i;

    degree = first_degree > second_degree ? first_degree : second_degree;

    for (i = 0; i <= degree; i++)
    {
        result[i] = 0;

        if (i <= first_degree)
        {
            result[i] += first[i];
        }

        if (i <= second_degree)
        {
            result[i] -= second[i];
        }
    }
```

```
    return degree;
}

int main(void)
{
    int first[MAX_DEGREE + 1] = {0};
    int second[MAX_DEGREE + 1] = {0};
    int result[MAX_DEGREE + 1] = {0};
    int first_degree;
    int second_degree;
    int result_degree;

    printf("First polynomial:\n");
    first_degree = read_polynomial(first);

    if (first_degree == -1)
    {
        return 0;
    }

    printf("Second polynomial:\n");
    second_degree = read_polynomial(second);

    if (second_degree == -1)
    {
        return 0;
    }

    result_degree = subtract_polynomials(first, first_degree, second, second_degree, result
    );

    printf("First polynomial: ");
    display_polynomial(first, first_degree);
    printf("Second polynomial: ");
    display_polynomial(second, second_degree);
    printf("Difference: ");
    display_polynomial(result, result_degree);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the degree of the polynomial: 2
Enter the coefficient of x^2: 5
Enter the coefficient of x^1: 3
Enter the coefficient of x^0: 1
Second polynomial:
Enter the degree of the polynomial: 1
Enter the coefficient of x^1: 2
Enter the coefficient of x^0: 1
First polynomial: 5x^2 + 3x + 1
Second polynomial: 2x + 1
Difference: 5x^2 + x
```

Program No: 20 | **Date:** 21-08-2026

Program Title: Multiply Two Polynomials Using Arrays

Description:

Multiplies two polynomials represented using coefficient arrays.

Author: Abhishek Vincent Roll No: 03 Date: 21-08-2026

Source Code:

```
#include <stdio.h>

#define MAX_DEGREE 20

int read_polynomial(int coefficients[])
{
    int degree;
    int i;

    printf("Enter the degree of the polynomial: ");
    scanf("%d", &degree);

    if (degree < 0 || degree > MAX_DEGREE)
    {
        printf("Invalid degree.\n");
        return -1;
    }

    for (i = degree; i >= 0; i--)
    {
        printf("Enter the coefficient of x^%d: ", i);
        scanf("%d", &coefficients[i]);
    }

    return degree;
}

void display_polynomial(int coefficients[], int degree)
{
    int i;
    int printed = 0;

    for (i = degree; i >= 0; i--)
    {
        if (coefficients[i] == 0)
        {
            continue;
        }

        if (printed == 1 && coefficients[i] > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficients[i] < 0)
        {
            printf(" - ");
        }
        else if (coefficients[i] < 0)
        {
            printf("-");
```

```
        }

        if (i == 0)
        {
            printf("%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
        }
        else if (i == 1)
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x");
            }
            else
            {
                printf("%dx", coefficients[i] < 0 ? -coefficients[i] : coefficients[i]);
            }
        }
        else
        {
            if (coefficients[i] == 1 || coefficients[i] == -1)
            {
                printf("x^%d", i);
            }
            else
            {
                printf("%dx^%d", coefficients[i] < 0 ? -coefficients[i] : coefficients[i],
    i);
            }
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

int multiply_polynomials(int first[], int first_degree, int second[], int second_degree,
    int result[])
{
    int degree;
    int i;
    int j;

    degree = first_degree + second_degree;

    for (i = 0; i <= degree; i++)
    {
        result[i] = 0;
    }

    for (i = 0; i <= first_degree; i++)
    {
        for (j = 0; j <= second_degree; j++)
        {
            result[i + j] += first[i] * second[j];
        }
    }
```

```
    return degree;
}

int main(void)
{
    int first[MAX_DEGREE + 1] = {0};
    int second[MAX_DEGREE + 1] = {0};
    int result[2 * MAX_DEGREE + 1] = {0};
    int first_degree;
    int second_degree;
    int result_degree;

    printf("First polynomial:\n");
    first_degree = read_polynomial(first);

    if (first_degree == -1)
    {
        return 0;
    }

    printf("Second polynomial:\n");
    second_degree = read_polynomial(second);

    if (second_degree == -1)
    {
        return 0;
    }

    result_degree = multiply_polynomials(first, first_degree, second, second_degree, result
    );

    printf("First polynomial: ");
    display_polynomial(first, first_degree);
    printf("Second polynomial: ");
    display_polynomial(second, second_degree);
    printf("Product: ");
    display_polynomial(result, result_degree);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the degree of the polynomial: 1
Enter the coefficient of x^1: 2
Enter the coefficient of x^0: 1
Second polynomial:
Enter the degree of the polynomial: 1
Enter the coefficient of x^1: 3
Enter the coefficient of x^0: 4
First polynomial: 2x + 1
Second polynomial: 3x + 4
Product: 6x^2 + 11x + 4
```

Program No: 21 | **Date:** 26-08-2026

Program Title: Date Structure

Description:

Defines a date structure and provides functions to read, display, and compare two dates for equality.

Author: Abhishek Vincent Roll No: 03 Date: 26-08-2026

Source Code:

```
#include <stdio.h>

typedef struct
{
    int day;
    int month;
    int year;
} Date;

void read_date(Date *date)
{
    printf("Enter the date (dd mm yyyy): ");
    scanf("%d %d %d", &date->day, &date->month, &date->year);
}

void display_date(Date date)
{
    printf("%02d/%02d/%04d\n", date.day, date.month, date.year);
}

int compare_dates(Date first, Date second)
{
    if (first.day == second.day &&
        first.month == second.month &&
        first.year == second.year)
    {
        return 1;
    }

    return 0;
}

int main(void)
{
    Date first;
    Date second;

    printf("First date:\n");
    read_date(&first);

    printf("Second date:\n");
    read_date(&second);

    printf("First date: ");
    display_date(first);

    printf("Second date: ");
    display_date(second);

    if (compare_dates(first, second) == 1)
```

```
    {
        printf("The dates are equal.\n");
    }
    else
    {
        printf("The dates are not equal.\n");
    }

    return 0;
}
```

Output Text:

```
First date:
Enter the date (dd mm yyyy): 15 8 2024
Second date:
Enter the date (dd mm yyyy): 15 8 2024
First date: 15/08/2024
Second date: 15/08/2024
The dates are equal.
```

Program No: 22 | **Date:** 26-08-2026

Program Title: Employee Structure

Description:

Defines an employee structure and provides functions to search, sort, and delete employee records.

Author: Abhishek Vincent Roll No: 03 Date: 26-08-2026

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_EMPLOYEES 50

typedef struct
{
    int eno;
    char ename[50];
    float esal;
    int dno;
} Employee;

void read_employees(Employee employees[], int count)
{
    int i;

    for (i = 0; i < count; i++)
    {
        printf("Employee %d\n", i + 1);
        printf("Enter employee number: ");
        scanf("%d", &employees[i].eno);
        printf("Enter employee name: ");
        scanf(" %49[^\n]", employees[i].ename);
        printf("Enter employee salary: ");
        scanf("%f", &employees[i].esal);
        printf("Enter department number: ");
        scanf("%d", &employees[i].dno);
    }
}

void display_employees(Employee employees[], int count)
{
    int i;

    printf("%-8s %-25s %-10s %-8s\n", "Eno", "Name", "Salary", "Dno");

    for (i = 0; i < count; i++)
    {
        printf("%-8d %-25s %-10.2f %-8d\n",
               employees[i].eno,
               employees[i].ename,
               employees[i].esal,
               employees[i].dno);
    }
}

void search_by_number(Employee employees[], int count, int number)
{
    int i;
```

```
    for (i = 0; i < count; i++)
    {
        if (employees[i].eno == number)
        {
            printf("Employee found: %d %s %.2f %d\n",
                   employees[i].eno,
                   employees[i].ename,
                   employees[i].esal,
                   employees[i].dno);
            return;
        }
    }

    printf("Employee %d not found.\n", number);
}

void sort_by_name(Employee employees[], int count)
{
    int i;
    int j;
    Employee temporary;

    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - i - 1; j++)
        {
            if (strcmp(employees[j].ename, employees[j + 1].ename) > 0)
            {
                temporary = employees[j];
                employees[j] = employees[j + 1];
                employees[j + 1] = temporary;
            }
        }
    }
}

void sort_by_salary(Employee employees[], int count)
{
    int i;
    int j;
    Employee temporary;

    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - i - 1; j++)
        {
            if (employees[j].esal < employees[j + 1].esal)
            {
                temporary = employees[j];
                employees[j] = employees[j + 1];
                employees[j + 1] = temporary;
            }
        }
    }
}

int delete_employee(Employee employees[], int count, int number)
{
    int i;
    int position = -1;

    for (i = 0; i < count; i++)
```

```
    {
        if (employees[i].eno == number)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Employee %d not found.\n", number);
        return count;
    }

    for (i = position; i < count - 1; i++)
    {
        employees[i] = employees[i + 1];
    }

    printf("Employee %d deleted.\n", number);
    return count - 1;
}

int main(void)
{
    Employee employees[MAX_EMPLOYEES];
    int count;

    printf("Enter the number of employees: ");
    scanf("%d", &count);

    if (count < 1 || count > MAX_EMPLOYEES)
    {
        printf("Invalid number of employees.\n");
        return 0;
    }

    read_employees(employees, count);

    printf("\nEmployee records:\n");
    display_employees(employees, count);

    printf("\nSearch by employee number 102:\n");
    search_by_number(employees, count, 102);

    printf("\nSorted by name:\n");
    sort_by_name(employees, count);
    display_employees(employees, count);

    printf("\nSorted by salary:\n");
    sort_by_salary(employees, count);
    display_employees(employees, count);

    printf("\nDelete employee number 102:\n");
    count = delete_employee(employees, count, 102);
    display_employees(employees, count);

    return 0;
}
```

Output Text:

```
Enter the number of employees: 3
```

```
Employee 1
Enter employee number: 101
Enter employee name: Alice
Enter employee salary: 50000
Enter department number: 1
Employee 2
Enter employee number: 102
Enter employee name: Bob
Enter employee salary: 60000
Enter department number: 2
Employee 3
Enter employee number: 103
Enter employee name: Carol
Enter employee salary: 55000
Enter department number: 1

Employee records:
Eno      Name                      Salary     Dno
101      Alice                     50000.00   1
102      Bob                       60000.00   2
103      Carol                     55000.00   1

Search by employee number 102:
Employee found: 102 Bob 60000.00 2

Sorted by name:
Eno      Name                      Salary     Dno
101      Alice                     50000.00   1
102      Bob                       60000.00   2
103      Carol                     55000.00   1

Sorted by salary:
Eno      Name                      Salary     Dno
102      Bob                       60000.00   2
103      Carol                     55000.00   1
101      Alice                     50000.00   1

Delete employee number 102:
Employee 102 deleted.
Eno      Name                      Salary     Dno
103      Carol                     55000.00   1
101      Alice                     50000.00   1
```

Program No: 23 | **Date:** 02-09-2026

Program Title: Read and Display a Polynomial Using Structures

Description:

Reads and displays a polynomial using an array of term structures.

Author: Abhishek Vincent Roll No: 03 Date: 02-09-2026

Source Code:

```
#include <stdio.h>

#define MAX_TERMS 50

typedef struct
{
    int coefficient;
    int exponent;
} Term;

typedef struct
{
    Term terms[MAX_TERMS];
    int count;
} Polynomial;

void sort_terms(Polynomial *polynomial)
{
    int i;
    int j;
    Term temporary;

    for (i = 0; i < polynomial->count - 1; i++)
    {
        for (j = 0; j < polynomial->count - i - 1; j++)
        {
            if (polynomial->terms[j].exponent < polynomial->terms[j + 1].exponent)
            {
                temporary = polynomial->terms[j];
                polynomial->terms[j] = polynomial->terms[j + 1];
                polynomial->terms[j + 1] = temporary;
            }
        }
    }
}

void read_polynomial(Polynomial *polynomial)
{
    int i;

    printf("Enter the number of terms: ");
    scanf("%d", &polynomial->count);

    if (polynomial->count < 1 || polynomial->count > MAX_TERMS)
    {
        printf("Invalid number of terms.\n");
        polynomial->count = 0;
        return;
    }
```

```
    for (i = 0; i < polynomial->count; i++)
    {
        printf("Term %d coefficient: ", i + 1);
        scanf("%d", &polynomial->terms[i].coefficient);
        printf("Term %d exponent: ", i + 1);
        scanf("%d", &polynomial->terms[i].exponent);
    }

    sort_terms(polynomial);
}

void display_polynomial(Polynomial polynomial)
{
    int i;
    int printed = 0;

    for (i = 0; i < polynomial.count; i++)
    {
        int coefficient = polynomial.terms[i].coefficient;
        int exponent = polynomial.terms[i].exponent;

        if (coefficient == 0)
        {
            continue;
        }

        if (printed == 1 && coefficient > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficient < 0)
        {
            printf(" - ");
        }
        else if (coefficient < 0)
        {
            printf("-");
        }

        if (coefficient < 0)
        {
            coefficient = -coefficient;
        }

        if (exponent == 0)
        {
            printf("%d", coefficient);
        }
        else if (exponent == 1)
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x");
        }
        else
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x^%d", exponent);
```

```
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

int main(void)
{
    Polynomial polynomial;

    read_polynomial(&polynomial);

    if (polynomial.count > 0)
    {
        printf("Polynomial: ");
        display_polynomial(polynomial);
    }

    return 0;
}
```

Output Text:

```
Enter the number of terms: 3
Term 1 coefficient: 2 3
Term 1 exponent: Term 2 coefficient: -3 1
Term 2 exponent: Term 3 coefficient: 4 0
Term 3 exponent: Polynomial: 2x^3 - 3x + 4
```

Program No: 24 | **Date:** 02-09-2026

Program Title: Add Two Polynomials Using Structures

Description:

Adds two polynomials represented using arrays of term structures.

Author: Abhishek Vincent Roll No: 03 Date: 02-09-2026

Source Code:

```
#include <stdio.h>

#define MAX_TERMS 50

typedef struct
{
    int coefficient;
    int exponent;
} Term;

typedef struct
{
    Term terms[MAX_TERMS];
    int count;
} Polynomial;

void sort_terms(Polynomial *polynomial)
{
    int i;
    int j;
    Term temporary;

    for (i = 0; i < polynomial->count - 1; i++)
    {
        for (j = 0; j < polynomial->count - i - 1; j++)
        {
            if (polynomial->terms[j].exponent < polynomial->terms[j + 1].exponent)
            {
                temporary = polynomial->terms[j];
                polynomial->terms[j] = polynomial->terms[j + 1];
                polynomial->terms[j + 1] = temporary;
            }
        }
    }
}

void read_polynomial(Polynomial *polynomial)
{
    int i;

    printf("Enter the number of terms: ");
    scanf("%d", &polynomial->count);

    if (polynomial->count < 1 || polynomial->count > MAX_TERMS)
    {
        printf("Invalid number of terms.\n");
        polynomial->count = 0;
        return;
    }
```

```
    for (i = 0; i < polynomial->count; i++)
    {
        printf("Term %d coefficient: ", i + 1);
        scanf("%d", &polynomial->terms[i].coefficient);
        printf("Term %d exponent: ", i + 1);
        scanf("%d", &polynomial->terms[i].exponent);
    }

    sort_terms(polynomial);
}

void display_polynomial(Polynomial polynomial)
{
    int i;
    int printed = 0;

    for (i = 0; i < polynomial.count; i++)
    {
        int coefficient = polynomial.terms[i].coefficient;
        int exponent = polynomial.terms[i].exponent;

        if (coefficient == 0)
        {
            continue;
        }

        if (printed == 1 && coefficient > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficient < 0)
        {
            printf(" - ");
        }
        else if (coefficient < 0)
        {
            printf("-");
        }

        if (coefficient < 0)
        {
            coefficient = -coefficient;
        }

        if (exponent == 0)
        {
            printf("%d", coefficient);
        }
        else if (exponent == 1)
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x");
        }
        else
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x^%d", exponent);
```

```
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

void combine_terms(Polynomial *polynomial)
{
    int i;
    int j;
    int k;

    sort_terms(polynomial);

    for (i = 0; i < polynomial->count; i++)
    {
        j = i + 1;

        while (j < polynomial->count)
        {
            if (polynomial->terms[i].exponent == polynomial->terms[j].exponent)
            {
                polynomial->terms[i].coefficient += polynomial->terms[j].coefficient;

                for (k = j; k < polynomial->count - 1; k++)
                {
                    polynomial->terms[k] = polynomial->terms[k + 1];
                }

                polynomial->count--;
            }
            else
            {
                j++;
            }
        }
    }

    i = 0;

    while (i < polynomial->count)
    {
        if (polynomial->terms[i].coefficient == 0)
        {
            for (k = i; k < polynomial->count - 1; k++)
            {
                polynomial->terms[k] = polynomial->terms[k + 1];
            }

            polynomial->count--;
        }
        else
        {
            i++;
        }
    }
```

```
}

void add_polynomials(Polynomial first, Polynomial second, Polynomial *result)
{
    int i;

    result->count = 0;

    for (i = 0; i < first.count; i++)
    {
        result->terms[result->count] = first.terms[i];
        result->count++;
    }

    for (i = 0; i < second.count; i++)
    {
        result->terms[result->count] = second.terms[i];
        result->count++;
    }

    combine_terms(result);
}

int main(void)
{
    Polynomial first;
    Polynomial second;
    Polynomial result;

    printf("First polynomial:\n");
    read_polynomial(&first);

    printf("Second polynomial:\n");
    read_polynomial(&second);

    if (first.count == 0 || second.count == 0)
    {
        return 0;
    }

    add_polynomials(first, second, &result);

    printf("First polynomial: ");
    display_polynomial(first);

    printf("Second polynomial: ");
    display_polynomial(second);

    printf("Sum: ");
    display_polynomial(result);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the number of terms: 2
Term 1 coefficient: 2
Term 1 exponent: 2
Term 2 coefficient: 1
Term 2 exponent: 0
Second polynomial:
```

```
Enter the number of terms: 2
Term 1 coefficient: 3
Term 1 exponent: 2
Term 2 coefficient: 4
Term 2 exponent: 0
First polynomial: 2x^2 + 1
Second polynomial: 3x^2 + 4
Sum: 5x^2 + 5
```

Program No: 25 | **Date:** 04-09-2026

Program Title: Subtract Two Polynomials Using Structures

Description:

Subtracts one polynomial from another using arrays of term structures.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>

#define MAX_TERMS 50

typedef struct
{
    int coefficient;
    int exponent;
} Term;

typedef struct
{
    Term terms[MAX_TERMS];
    int count;
} Polynomial;

void sort_terms(Polynomial *polynomial)
{
    int i;
    int j;
    Term temporary;

    for (i = 0; i < polynomial->count - 1; i++)
    {
        for (j = 0; j < polynomial->count - i - 1; j++)
        {
            if (polynomial->terms[j].exponent < polynomial->terms[j + 1].exponent)
            {
                temporary = polynomial->terms[j];
                polynomial->terms[j] = polynomial->terms[j + 1];
                polynomial->terms[j + 1] = temporary;
            }
        }
    }
}

void read_polynomial(Polynomial *polynomial)
{
    int i;

    printf("Enter the number of terms: ");
    scanf("%d", &polynomial->count);

    if (polynomial->count < 1 || polynomial->count > MAX_TERMS)
    {
        printf("Invalid number of terms.\n");
        polynomial->count = 0;
        return;
    }
```

```
    for (i = 0; i < polynomial->count; i++)
    {
        printf("Term %d coefficient: ", i + 1);
        scanf("%d", &polynomial->terms[i].coefficient);
        printf("Term %d exponent: ", i + 1);
        scanf("%d", &polynomial->terms[i].exponent);
    }

    sort_terms(polynomial);
}

void display_polynomial(Polynomial polynomial)
{
    int i;
    int printed = 0;

    for (i = 0; i < polynomial.count; i++)
    {
        int coefficient = polynomial.terms[i].coefficient;
        int exponent = polynomial.terms[i].exponent;

        if (coefficient == 0)
        {
            continue;
        }

        if (printed == 1 && coefficient > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficient < 0)
        {
            printf(" - ");
        }
        else if (coefficient < 0)
        {
            printf("-");
        }

        if (coefficient < 0)
        {
            coefficient = -coefficient;
        }

        if (exponent == 0)
        {
            printf("%d", coefficient);
        }
        else if (exponent == 1)
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x");
        }
        else
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x^%d", exponent);
```

```
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

void combine_terms(Polynomial *polynomial)
{
    int i;
    int j;
    int k;

    sort_terms(polynomial);

    for (i = 0; i < polynomial->count; i++)
    {
        j = i + 1;

        while (j < polynomial->count)
        {
            if (polynomial->terms[i].exponent == polynomial->terms[j].exponent)
            {
                polynomial->terms[i].coefficient += polynomial->terms[j].coefficient;

                for (k = j; k < polynomial->count - 1; k++)
                {
                    polynomial->terms[k] = polynomial->terms[k + 1];
                }

                polynomial->count--;
            }
            else
            {
                j++;
            }
        }
    }

    i = 0;

    while (i < polynomial->count)
    {
        if (polynomial->terms[i].coefficient == 0)
        {
            for (k = i; k < polynomial->count - 1; k++)
            {
                polynomial->terms[k] = polynomial->terms[k + 1];
            }

            polynomial->count--;
        }
        else
        {
            i++;
        }
    }
```

```
}

void subtract_polynomials(Polynomial first, Polynomial second, Polynomial *result)
{
    int i;

    result->count = 0;

    for (i = 0; i < first.count; i++)
    {
        result->terms[result->count] = first.terms[i];
        result->count++;
    }

    for (i = 0; i < second.count; i++)
    {
        result->terms[result->count] = second.terms[i];
        result->terms[result->count].coefficient = -second.terms[i].coefficient;
        result->count++;
    }

    combine_terms(result);
}

int main(void)
{
    Polynomial first;
    Polynomial second;
    Polynomial result;

    printf("First polynomial:\n");
    read_polynomial(&first);

    printf("Second polynomial:\n");
    read_polynomial(&second);

    if (first.count == 0 || second.count == 0)
    {
        return 0;
    }

    subtract_polynomials(first, second, &result);

    printf("First polynomial: ");
    display_polynomial(first);

    printf("Second polynomial: ");
    display_polynomial(second);

    printf("Difference: ");
    display_polynomial(result);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the number of terms: 2
Term 1 coefficient: 5
Term 1 exponent: 2
Term 2 coefficient: 3
Term 2 exponent: 0
```

```
Second polynomial:
Enter the number of terms: 2
Term 1 coefficient: 2
Term 1 exponent: 2
Term 2 coefficient: 1
Term 2 exponent: 0
First polynomial: 5x^2 + 3
Second polynomial: 2x^2 + 1
Difference: 3x^2 + 2
```

Program No: 26 | **Date:** 04-09-2026

Program Title: Multiply Two Polynomials Using Structures

Description:

Multiplies two polynomials represented using arrays of term structures.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>

#define MAX_TERMS 50

typedef struct
{
    int coefficient;
    int exponent;
} Term;

typedef struct
{
    Term terms[MAX_TERMS];
    int count;
} Polynomial;

void sort_terms(Polynomial *polynomial)
{
    int i;
    int j;
    Term temporary;

    for (i = 0; i < polynomial->count - 1; i++)
    {
        for (j = 0; j < polynomial->count - i - 1; j++)
        {
            if (polynomial->terms[j].exponent < polynomial->terms[j + 1].exponent)
            {
                temporary = polynomial->terms[j];
                polynomial->terms[j] = polynomial->terms[j + 1];
                polynomial->terms[j + 1] = temporary;
            }
        }
    }
}

void read_polynomial(Polynomial *polynomial)
{
    int i;

    printf("Enter the number of terms: ");
    scanf("%d", &polynomial->count);

    if (polynomial->count < 1 || polynomial->count > MAX_TERMS)
    {
        printf("Invalid number of terms.\n");
        polynomial->count = 0;
        return;
    }
```

```
    for (i = 0; i < polynomial->count; i++)
    {
        printf("Term %d coefficient: ", i + 1);
        scanf("%d", &polynomial->terms[i].coefficient);
        printf("Term %d exponent: ", i + 1);
        scanf("%d", &polynomial->terms[i].exponent);
    }

    sort_terms(polynomial);
}

void display_polynomial(Polynomial polynomial)
{
    int i;
    int printed = 0;

    for (i = 0; i < polynomial.count; i++)
    {
        int coefficient = polynomial.terms[i].coefficient;
        int exponent = polynomial.terms[i].exponent;

        if (coefficient == 0)
        {
            continue;
        }

        if (printed == 1 && coefficient > 0)
        {
            printf(" + ");
        }
        else if (printed == 1 && coefficient < 0)
        {
            printf(" - ");
        }
        else if (coefficient < 0)
        {
            printf("-");
        }

        if (coefficient < 0)
        {
            coefficient = -coefficient;
        }

        if (exponent == 0)
        {
            printf("%d", coefficient);
        }
        else if (exponent == 1)
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x");
        }
        else
        {
            if (coefficient != 1)
            {
                printf("%d", coefficient);
            }
            printf("x^%d", exponent);
```

```
        }

        printed = 1;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

void combine_terms(Polynomial *polynomial)
{
    int i;
    int j;
    int k;

    sort_terms(polynomial);

    for (i = 0; i < polynomial->count; i++)
    {
        j = i + 1;

        while (j < polynomial->count)
        {
            if (polynomial->terms[i].exponent == polynomial->terms[j].exponent)
            {
                polynomial->terms[i].coefficient += polynomial->terms[j].coefficient;

                for (k = j; k < polynomial->count - 1; k++)
                {
                    polynomial->terms[k] = polynomial->terms[k + 1];
                }

                polynomial->count--;
            }
            else
            {
                j++;
            }
        }
    }

    i = 0;

    while (i < polynomial->count)
    {
        if (polynomial->terms[i].coefficient == 0)
        {
            for (k = i; k < polynomial->count - 1; k++)
            {
                polynomial->terms[k] = polynomial->terms[k + 1];
            }

            polynomial->count--;
        }
        else
        {
            i++;
        }
    }
```

```
}

void multiply_polynomials(Polynomial first, Polynomial second, Polynomial *result)
{
    int i;
    int j;

    result->count = 0;

    for (i = 0; i < first.count; i++)
    {
        for (j = 0; j < second.count; j++)
        {
            result->terms[result->count].coefficient =
                first.terms[i].coefficient * second.terms[j].coefficient;
            result->terms[result->count].exponent =
                first.terms[i].exponent + second.terms[j].exponent;
            result->count++;
        }
    }

    combine_terms(result);
}

int main(void)
{
    Polynomial first;
    Polynomial second;
    Polynomial result;

    printf("First polynomial:\n");
    read_polynomial(&first);

    printf("Second polynomial:\n");
    read_polynomial(&second);

    if (first.count == 0 || second.count == 0)
    {
        return 0;
    }

    multiply_polynomials(first, second, &result);

    printf("First polynomial: ");
    display_polynomial(first);

    printf("Second polynomial: ");
    display_polynomial(second);

    printf("Product: ");
    display_polynomial(result);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the number of terms: 2
Term 1 coefficient: 2
Term 1 exponent: 1
Term 2 coefficient: 1
Term 2 exponent: 0
```

```
Second polynomial:
Enter the number of terms: 2
Term 1 coefficient: 3
Term 1 exponent: 1
Term 2 coefficient: 4
Term 2 exponent: 0
First polynomial: 2x + 1
Second polynomial: 3x + 4
Product: 6x^2 + 11x + 4
```

Program No: 27 | **Date:** 09-09-2026

Program Title: Reverse a String

Description:

Reverses a string by printing it backwards and by reversing it in the same array.

Author: Abhishek Vincent Roll No: 03 Date: 09-09-2026

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_LENGTH 100

void read_string(char text[])
{
    printf("Enter a string: ");
    scanf(" %99[^\n]", text);
}

void print_reverse(char text[])
{
    int i;

    printf("Reverse order: ");

    for (i = (int) strlen(text) - 1; i >= 0; i--)
    {
        printf("%c", text[i]);
    }

    printf("\n");
}

void reverse_in_place(char text[])
{
    int i;
    int j;
    char temporary;

    i = 0;
    j = (int) strlen(text) - 1;

    while (i < j)
    {
        temporary = text[i];
        text[i] = text[j];
        text[j] = temporary;
        i++;
        j--;
    }
}

int main(void)
{
    char text[MAX_LENGTH];

    read_string(text);
```

```
    print_reverse(text);

    reverse_in_place(text);
    printf("Reversed string: %s\n", text);

    return 0;
}
```

Output Text:

```
Enter a string: programming
Reverse order: gnimmargorp
Reversed string: gnimmargorp
```

Program No: 28 | **Date:** 09-09-2026

Program Title: Sort n Strings in Ascending Order

Description:

Reads n strings and displays them in ascending order.

Author: Abhishek Vincent Roll No: 03 Date: 09-09-2026

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_STRINGS 20
#define MAX_LENGTH 50

int read_strings(char strings[][MAX_LENGTH])
{
    int count;
    int i;

    printf("Enter the number of strings: ");
    scanf("%d", &count);

    if (count < 1 || count > MAX_STRINGS)
    {
        printf("Invalid number of strings.\n");
        return 0;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter string %d: ", i + 1);
        scanf(" %49[^\n]", strings[i]);
    }

    return count;
}

void display_strings(char strings[][MAX_LENGTH], int count)
{
    int i;

    for (i = 0; i < count; i++)
    {
        printf("%s\n", strings[i]);
    }
}

void sort_strings(char strings[][MAX_LENGTH], int count)
{
    int i;
    int j;
    char temporary[MAX_LENGTH];

    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - i - 1; j++)
        {
            if (strcmp(strings[j], strings[j + 1]) > 0)
```

```
            {
                strcpy(temporary, strings[j]);
                strcpy(strings[j], strings[j + 1]);
                strcpy(strings[j + 1], temporary);
            }
        }
    }
}

int main(void)
{
    char strings[MAX_STRINGS][MAX_LENGTH];
    int count;

    count = read_strings(strings);

    if (count == 0)
    {
        return 0;
    }

    printf("Before sorting:\n");
    display_strings(strings, count);

    sort_strings(strings, count);

    printf("After sorting:\n");
    display_strings(strings, count);

    return 0;
}
```

Output Text:

```
Enter the number of strings: 4
Enter string 1: banana
Enter string 2: apple
Enter string 3: cherry
Enter string 4: date
Before sorting:
banana
apple
cherry
date
After sorting:
apple
banana
cherry
date
```

Program No: 29 | **Date:** 16-09-2026

Program Title: Queue Using an Array

Description:

Demonstrates enqueue, dequeue, and display operations on a linear queue using an array.

Author: Abhishek Vincent Roll No: 03 Date: 16-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = 0;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        printf("Queue is full.\n");
        return;
    }

    rear++;
    queue[rear] = value;
}

int dequeue(void)
{
    int value;

    if (front > rear)
    {
        printf("Queue is empty.\n");
        return -1;
    }

    value = queue[front];
    front++;

    return value;
}

void display(void)
{
    int i;

    if (front > rear)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue: ");

    for (i = front; i <= rear; i++)
    {
```

```
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main(void)
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    printf("Dequeued: %d\n", dequeue());
    display();

    enqueue(40);
    display();

    return 0;
}
```

Output Text:

```
Queue: 10 20 30
Dequeued: 10
Queue: 20 30
Queue: 20 30 40
```

Program No: 30 | **Date:** 16-09-2026

Program Title: Circular Queue Using an Array

Description:

Demonstrates enqueue, dequeue, and display operations on a circular queue using an array.

Author: Abhishek Vincent Roll No: 03 Date: 16-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = 0;
int rear = -1;
int count = 0;

void enqueue(int value)
{
    if (count == MAX)
    {
        printf("Queue is full.\n");
        return;
    }

    rear = (rear + 1) % MAX;
    queue[rear] = value;
    count++;
}

int dequeue(void)
{
    int value;

    if (count == 0)
    {
        printf("Queue is empty.\n");
        return -1;
    }

    value = queue[front];
    front = (front + 1) % MAX;
    count--;

    return value;
}

void display(void)
{
    int i;

    if (count == 0)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue: ");
```

```
    for (i = 0; i < count; i++)
    {
        printf("%d ", queue[(front + i) % MAX]);
    }

    printf("\n");
}

int main(void)
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);

    display();

    printf("Dequeued: %d\n", dequeue());
    printf("Dequeued: %d\n", dequeue());

    enqueue(50);
    enqueue(60);

    display();

    return 0;
}
```

Output Text:

```
Queue: 10 20 30 40
Dequeued: 10
Dequeued: 20
Queue: 30 40 50 60
```

Program No: 31 | **Date:** 04-09-2026

Program Title: Dynamic Memory Allocation Functions

Description:

Demonstrates the use of malloc, calloc, and free for dynamic memory allocation.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count = 5;
    int i;
    int *malloc_array;
    int *calloc_array;

    malloc_array = malloc(count * sizeof(int));

    if (malloc_array == NULL)
    {
        printf("malloc failed.\n");
        return 1;
    }

    printf("Values allocated with malloc:\n");

    for (i = 0; i < count; i++)
    {
        malloc_array[i] = (i + 1) * 10;
        printf("%d ", malloc_array[i]);
    }

    printf("\n");
    free(malloc_array);
    printf("Memory allocated with malloc has been freed.\n");

    calloc_array = calloc(count, sizeof(int));

    if (calloc_array == NULL)
    {
        printf("calloc failed.\n");
        return 1;
    }

    printf("Values allocated with calloc:\n");

    for (i = 0; i < count; i++)
    {
        printf("%d ", calloc_array[i]);
    }

    printf("\n");
    free(calloc_array);
    printf("Memory allocated with calloc has been freed.\n");

    return 0;
```

```
}
```

Output Text:

```
Values allocated with malloc:
10 20 30 40 50
Memory allocated with malloc has been freed.
Values allocated with calloc:
0 0 0 0 0
Memory allocated with calloc has been freed.
```

Program No: 32 | **Date:** 04-09-2026

Program Title: Mean Using malloc

Description:

Uses malloc to read n integers and computes their mean.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count;
    int i;
    int sum = 0;
    float mean;
    int *numbers;

    printf("Enter the number of integers: ");
    scanf("%d", &count);

    if (count < 1)
    {
        printf("Invalid number of integers.\n");
        return 0;
    }

    numbers = malloc(count * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter integer %d: ", i + 1);
        scanf("%d", &numbers[i]);
        sum += numbers[i];
    }

    mean = (float) sum / count;

    printf("Mean = %.2f\n", mean);

    free(numbers);

    return 0;
}
```

Output Text:

```
Enter the number of integers: 5
Enter integer 1: 10
Enter integer 2: 20
```

```
Enter integer 3: 30
Enter integer 4: 40
Enter integer 5: 50
Mean = 30.00
```

Program No: 33 | **Date:** 04-09-2026

Program Title: Mode Using calloc

Description:

Uses calloc to count frequencies and find the mode of n numbers.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count;
    int i;
    int maximum;
    int mode;
    int highest_frequency = 0;
    int *numbers;
    int *frequency;

    printf("Enter the number of values: ");
    scanf("%d", &count);

    if (count < 1)
    {
        printf("Invalid number of values.\n");
        return 0;
    }

    numbers = malloc(count * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter value %d: ", i + 1);
        scanf("%d", &numbers[i]);
    }

    maximum = numbers[0];

    for (i = 1; i < count; i++)
    {
        if (numbers[i] > maximum)
        {
            maximum = numbers[i];
        }
    }

    frequency = calloc(maximum + 1, sizeof(int));

    if (frequency == NULL)
    {
```

```
        printf("Memory allocation failed.\n");
        free(numbers);
        return 1;
    }

    for (i = 0; i < count; i++)
    {
        frequency[numbers[i]]++;
    }

    mode = 0;

    for (i = 0; i <= maximum; i++)
    {
        if (frequency[i] > highest_frequency)
        {
            highest_frequency = frequency[i];
            mode = i;
        }
    }

    printf("Mode = %d (frequency %d)\n", mode, highest_frequency);

    free(numbers);
    free(frequency);

    return 0;
}
```

Output Text:

```
Enter the number of values: 8
Enter value 1: 1
Enter value 2: 2
Enter value 3: 2
Enter value 4: 3
Enter value 5: 3
Enter value 6: 3
Enter value 7: 4
Enter value 8: 4
Mode = 3 (frequency 3)
```

Program No: 34 | **Date:** 04-09-2026

Program Title: Array of Books Using Pointers

Description:

Creates an array of book structures using a pointer and reads and displays the books through pointers.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_LENGTH 50

typedef struct
{
    char author_name[MAX_LENGTH];
    char book_name[MAX_LENGTH];
} Book;

void read_books(Book *books, int count)
{
    int i;

    for (i = 0; i < count; i++)
    {
        printf("Book %d\n", i + 1);
        printf("Enter author name: ");
        scanf(" %49[^\n]", books[i].author_name);
        printf("Enter book name: ");
        scanf(" %49[^\n]", books[i].book_name);
    }
}

void display_books(Book *books, int count)
{
    int i;

    printf("%-25s %-25s\n", "Author", "Book");

    for (i = 0; i < count; i++)
    {
        printf("%-25s %-25s\n",
               (books + i)->author_name,
               (books + i)->book_name);
    }
}

int main(void)
{
    Book *books;
    int count;

    printf("Enter the number of books: ");
    scanf("%d", &count);

    if (count < 1)
    {
        printf("Invalid number of books.\n");
```

```
        return 0;
    }

    books = malloc(count * sizeof(Book));

    if (books == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    read_books(books, count);
    display_books(books, count);

    free(books);

    return 0;
}
```

Output Text:

```
Enter the number of books: 3
Book 1
Enter author name: Dennis Ritchie
Enter book name: The C Programming Language
Book 2
Enter author name: Brian Kernighan
Enter book name: The Unix Programming Environment
Book 3
Enter author name: Donald Knuth
Enter book name: The Art of Computer Programming
Author                  Book
Dennis Ritchie          The C Programming Language
Brian Kernighan         The Unix Programming Environment
Donald Knuth            The Art of Computer Programming
```

Program No: 35 | **Date:** 04-09-2026

Program Title: Variable-Length String Using realloc

Description:

Uses realloc to grow a character buffer and read a string of any length.

Author: Abhishek Vincent Roll No: 03 Date: 04-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int capacity = 2;
    int length = 0;
    char character;
    char *text;
    char *resized;

    text = malloc(capacity * sizeof(char));

    if (text == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter a string of any length: ");

    while ((character = (char) getchar()) != '\n' && character != EOF)
    {
        if (length + 1 >= capacity)
        {
            capacity = capacity * 2;
            resized = realloc(text, capacity * sizeof(char));

            if (resized == NULL)
            {
                printf("Memory reallocation failed.\n");
                free(text);
                return 1;
            }

            text = resized;
        }

        text[length] = character;
        length++;
    }

    text[length] = '\0';

    printf("You entered: %s\n", text);
    printf("Length: %d\n", length);

    free(text);

    return 0;
```

```
}
```

Output Text:

```
Enter a string of any length: This string is deliberately long so that the buffer has to
    grow several times.
You entered: This string is deliberately long so that the buffer has to grow several times.
Length: 78
```

Program No: 36 | **Date:** 11-09-2026

Program Title: Read and Display a Sparse Matrix

Description:

Reads a sparse matrix in triplet form and displays it in triplet and full form.

Author: Abhishek Vincent Roll No: 03 Date: 11-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 10
#define MAX_TERMS 50

typedef struct
{
    int row;
    int column;
    int value;
} Element;

int read_sparse(Element elements[], int *rows, int *columns)
{
    int count;
    int i;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", rows, columns);

    printf("Enter the number of non-zero elements: ");
    scanf("%d", &count);

    if (count < 1 || count > MAX_TERMS)
    {
        printf("Invalid number of elements.\n");
        return 0;
    }

    for (i = 0; i < count; i++)
    {
        printf("Element %d (row column value): ", i + 1);
        scanf("%d %d %d",
              &elements[i].row,
              &elements[i].column,
              &elements[i].value);
    }

    return count;
}

void display_triplets(Element elements[], int count)
{
    int i;

    printf("%-6s %-8s %-6s\n", "Row", "Column", "Value");

    for (i = 0; i < count; i++)
    {
        printf("%-6d %-8d %-6d\n",
```

```
                elements[i].row,
                elements[i].column,
                elements[i].value);
    }
}

void display_matrix(Element elements[], int count, int rows, int columns)
{
    int matrix[MAX][MAX] = {0};
    int i;
    int j;

    for (i = 0; i < count; i++)
    {
        matrix[elements[i].row][elements[i].column] = elements[i].value;
    }

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main(void)
{
    Element elements[MAX_TERMS];
    int rows;
    int columns;
    int count;

    count = read_sparse(elements, &rows, &columns);

    if (count == 0)
    {
        return 0;
    }

    printf("Sparse matrix in triplet form:\n");
    display_triplets(elements, count);

    printf("Sparse matrix in full form:\n");
    display_matrix(elements, count, rows, columns);

    return 0;
}
```

Output Text:

```
Enter the number of rows and columns: 3 3
Enter the number of non-zero elements: 4
Element 1 (row column value): 0 0 5
Element 2 (row column value): 0 2 3
Element 3 (row column value): 2 1 7
Element 4 (row column value): 1 0 2
Sparse matrix in triplet form:
Row    Column   Value
0      0        5
0      2        3
2      1        7
```

```
1      0        2
Sparse matrix in full form:
5 0 3
2 0 0
0 7 0
```

Program No: 37 | **Date:** 11-09-2026

Program Title: Add Two Sparse Matrices

Description:

Adds two sparse matrices represented in triplet form.

Author: Abhishek Vincent Roll No: 03 Date: 11-09-2026

Source Code:

```
#include <stdio.h>

#define MAX_TERMS 100

typedef struct
{
    int row;
    int column;
    int value;
} Element;

int read_sparse(Element elements[], int *rows, int *columns)
{
    int count;
    int i;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", rows, columns);

    printf("Enter the number of non-zero elements: ");
    scanf("%d", &count);

    if (count < 0 || count > MAX_TERMS / 2)
    {
        printf("Invalid number of elements.\n");
        return -1;
    }

    for (i = 0; i < count; i++)
    {
        printf("Element %d (row column value): ", i + 1);
        scanf("%d %d %d",
              &elements[i].row,
              &elements[i].column,
              &elements[i].value);
    }

    return count;
}

void sort_elements(Element elements[], int count)
{
    int i;
    int j;
    Element temporary;

    for (i = 0; i < count - 1; i++)
    {
        for (j = 0; j < count - i - 1; j++)
        {
```

```
            if (elements[j].row > elements[j + 1].row ||
                (elements[j].row == elements[j + 1].row &&
                 elements[j].column > elements[j + 1].column))
            {
                temporary = elements[j];
                elements[j] = elements[j + 1];
                elements[j + 1] = temporary;
            }
        }
    }
}

void combine_elements(Element elements[], int *count)
{
    int i;
    int j;
    int k;

    sort_elements(elements, *count);

    for (i = 0; i < *count; i++)
    {
        j = i + 1;

        while (j < *count)
        {
            if (elements[i].row == elements[j].row &&
                elements[i].column == elements[j].column)
            {
                elements[i].value += elements[j].value;

                for (k = j; k < *count - 1; k++)
                {
                    elements[k] = elements[k + 1];
                }

                (*count)--;
            }
            else
            {
                j++;
            }
        }
    }

    i = 0;

    while (i < *count)
    {
        if (elements[i].value == 0)
        {
            for (k = i; k < *count - 1; k++)
            {
                elements[k] = elements[k + 1];
            }

            (*count)--;
        }
        else
        {
            i++;
        }
    }
```

```
}

int add_sparse(Element first[], int first_count,
               Element second[], int second_count,
               Element result[])
{
    int i;
    int count = 0;

    for (i = 0; i < first_count; i++)
    {
        result[count] = first[i];
        count++;
    }

    for (i = 0; i < second_count; i++)
    {
        result[count] = second[i];
        count++;
    }

    combine_elements(result, &count);

    return count;
}

void display_triplets(Element elements[], int count)
{
    int i;

    printf("%-6s %-8s %-6s\n", "Row", "Column", "Value");

    for (i = 0; i < count; i++)
    {
        printf("%-6d %-8d %-6d\n",
               elements[i].row,
               elements[i].column,
               elements[i].value);
    }
}

int main(void)
{
    Element first[MAX_TERMS / 2];
    Element second[MAX_TERMS / 2];
    Element result[MAX_TERMS];
    int first_rows;
    int first_columns;
    int second_rows;
    int second_columns;
    int first_count;
    int second_count;
    int result_count;

    printf("First sparse matrix:\n");
    first_count = read_sparse(first, &first_rows, &first_columns);

    printf("Second sparse matrix:\n");
    second_count = read_sparse(second, &second_rows, &second_columns);

    if (first_count < 0 || second_count < 0)
    {
        return 0;
```

```
    }

    if (first_rows != second_rows || first_columns != second_columns)
    {
        printf("Matrix dimensions do not match.\n");
        return 0;
    }

    result_count = add_sparse(first, first_count, second, second_count, result);

    printf("Sum in triplet form:\n");
    display_triplets(result, result_count);

    return 0;
}
```

Output Text:

```
First sparse matrix:
Enter the number of rows and columns: 3 3
Enter the number of non-zero elements: 3
Element 1 (row column value): 0 0 5
Element 2 (row column value): 0 2 3
Element 3 (row column value): 2 1 7
Second sparse matrix:
Enter the number of rows and columns: 3 3
Enter the number of non-zero elements: 3
Element 1 (row column value): 0 0 2
Element 2 (row column value): 1 1 4
Element 3 (row column value): 2 1 -7
Sum in triplet form:
Row    Column   Value
0      0        7
0      2        3
1      1        4
```

Program No: 38 | **Date:** 11-09-2026

Program Title: Multiply Two Sparse Matrices

Description:

Multiplies two sparse matrices by converting them to dense form and back to triplets.

Author: Abhishek Vincent Roll No: 03 Date: 11-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 10
#define MAX_TERMS 100

typedef struct
{
    int row;
    int column;
    int value;
} Element;

int read_sparse(Element elements[], int *rows, int *columns)
{
    int count;
    int i;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", rows, columns);

    printf("Enter the number of non-zero elements: ");
    scanf("%d", &count);

    if (count < 0 || count > MAX_TERMS)
    {
        printf("Invalid number of elements.\n");
        return -1;
    }

    for (i = 0; i < count; i++)
    {
        printf("Element %d (row column value): ", i + 1);
        scanf("%d %d %d",
              &elements[i].row,
              &elements[i].column,
              &elements[i].value);
    }

    return count;
}

void to_dense(Element elements[], int count, int matrix[MAX][MAX])
{
    int i;
    int j;

    for (i = 0; i < MAX; i++)
    {
        for (j = 0; j < MAX; j++)
        {
```

```
            matrix[i][j] = 0;
        }
    }

    for (i = 0; i < count; i++)
    {
        matrix[elements[i].row][elements[i].column] = elements[i].value;
    }
}

int to_sparse(int matrix[MAX][MAX], int rows, int columns, Element result[])
{
    int i;
    int j;
    int count = 0;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            if (matrix[i][j] != 0)
            {
                result[count].row = i;
                result[count].column = j;
                result[count].value = matrix[i][j];
                count++;
            }
        }
    }

    return count;
}

void display_dense(int matrix[MAX][MAX], int rows, int columns)
{
    int i;
    int j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

void display_triplets(Element elements[], int count)
{
    int i;

    printf("%-6s %-8s %-6s\n", "Row", "Column", "Value");

    for (i = 0; i < count; i++)
    {
        printf("%-6d %-8d %-6d\n",
               elements[i].row,
               elements[i].column,
               elements[i].value);
    }
}
```

```
int main(void)
{
    Element first[MAX_TERMS];
    Element second[MAX_TERMS];
    Element result[MAX_TERMS];
    int first_dense[MAX][MAX];
    int second_dense[MAX][MAX];
    int product[MAX][MAX];
    int first_rows;
    int first_columns;
    int second_rows;
    int second_columns;
    int first_count;
    int second_count;
    int result_count;
    int i;
    int j;
    int k;

    printf("First sparse matrix:\n");
    first_count = read_sparse(first, &first_rows, &first_columns);

    printf("Second sparse matrix:\n");
    second_count = read_sparse(second, &second_rows, &second_columns);

    if (first_count < 0 || second_count < 0)
    {
        return 0;
    }

    if (first_columns != second_rows)
    {
        printf("Matrix multiplication is not possible.\n");
        return 0;
    }

    to_dense(first, first_count, first_dense);
    to_dense(second, second_count, second_dense);

    for (i = 0; i < first_rows; i++)
    {
        for (j = 0; j < second_columns; j++)
        {
            product[i][j] = 0;

            for (k = 0; k < first_columns; k++)
            {
                product[i][j] += first_dense[i][k] * second_dense[k][j];
            }
        }
    }

    printf("Product in full form:\n");
    display_dense(product, first_rows, second_columns);

    result_count = to_sparse(product, first_rows, second_columns, result);

    printf("Product in triplet form:\n");
    display_triplets(result, result_count);

    return 0;
}
```

Output Text:

```
First sparse matrix:
Enter the number of rows and columns: 2 3
Enter the number of non-zero elements: 4
Element 1 (row column value): 0 0 1
Element 2 (row column value): 0 1 2
Element 3 (row column value): 1 1 3
Element 4 (row column value): 1 2 4
Second sparse matrix:
Enter the number of rows and columns: 3 2
Enter the number of non-zero elements: 3
Element 1 (row column value): 0 0 5
Element 2 (row column value): 1 1 6
Element 3 (row column value): 2 0 7
Product in full form:
5 12
28 18
Product in triplet form:
Row    Column   Value
0      0        5
0      1        12
1      0        28
1      1        18
```

Program No: 39 | **Date:** 18-09-2026

Program Title: Heapsort

Description:

Sorts an array in ascending order using heapsort.

Author: Abhishek Vincent Roll No: 03 Date: 18-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void swap(int *first, int *second)
{
    int temporary = *first;
    *first = *second;
    *second = temporary;
}

void heapify(int array[], int size, int root)
{
    int largest = root;
    int left = 2 * root + 1;
    int right = 2 * root + 2;

    if (left < size && array[left] > array[largest])
    {
        largest = left;
    }

    if (right < size && array[right] > array[largest])
    {
        largest = right;
    }

    if (largest != root)
    {
        swap(&array[root], &array[largest]);
        heapify(array, size, largest);
    }
}

void heap_sort(int array[], int size)
{
    int i;

    for (i = size / 2 - 1; i >= 0; i--)
    {
        heapify(array, size, i);
    }

    for (i = size - 1; i > 0; i--)
    {
        swap(&array[0], &array[i]);
        heapify(array, i, 0);
    }
}
```

```
void display(int array[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }

    printf("\n");
}

int main(void)
{
    int array[MAX];
    int size;
    int i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
    }

    for (i = 0; i < size; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }

    printf("Before sorting: ");
    display(array, size);

    heap_sort(array, size);

    printf("After sorting: ");
    display(array, size);

    return 0;
}
```

Output Text:

```
Enter the size of the array: 6
Enter element 1: 12
Enter element 2: 11
Enter element 3: 13
Enter element 4: 5
Enter element 5: 6
Enter element 6: 7
Before sorting: 12 11 13 5 6 7
After sorting: 5 6 7 11 12 13
```

Program No: 40 | **Date:** 18-09-2026

Program Title: Priority Queue Using an Array

Description:

Demonstrates insertion, deletion of the highest-priority element, and display on a priority queue.

Author: Abhishek Vincent Roll No: 03 Date: 18-09-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

typedef struct
{
    int value;
    int priority;
} Item;

Item queue[MAX];
int count = 0;

void insert(int value, int priority)
{
    if (count == MAX)
    {
        printf("Priority queue is full.\n");
        return;
    }

    queue[count].value = value;
    queue[count].priority = priority;
    count++;
}

int delete_highest(void)
{
    int i;
    int highest = 0;
    int value;

    if (count == 0)
    {
        printf("Priority queue is empty.\n");
        return -1;
    }

    for (i = 1; i < count; i++)
    {
        if (queue[i].priority > queue[highest].priority)
        {
            highest = i;
        }
    }

    value = queue[highest].value;

    for (i = highest; i < count - 1; i++)
    {
```

```
        queue[i] = queue[i + 1];
    }

    count--;

    return value;
}

void display(void)
{
    int i;

    printf("%-8s %-8s\n", "Value", "Priority");

    for (i = 0; i < count; i++)
    {
        printf("%-8d %-8d\n", queue[i].value, queue[i].priority);
    }
}

int main(void)
{
    insert(10, 2);
    insert(20, 5);
    insert(30, 1);
    insert(40, 4);

    printf("Priority queue:\n");
    display();

    printf("Deleted: %d\n", delete_highest());
    display();

    printf("Deleted: %d\n", delete_highest());
    display();

    return 0;
}
```

Output Text:

```
Priority queue:
Value    Priority
10       2
20       5
30       1
40       4
Deleted: 20
Value    Priority
10       2
30       1
40       4
Deleted: 40
Value    Priority
10       2
30       1
```

Program No: 41 | **Date:** 25-09-2026

Program Title: Linked List Creation and Display

Description:

Creates a singly linked list and displays its elements.

Author: Abhishek Vincent Roll No: 03 Date: 25-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->next = NULL;

    return node;
}

void insert_end(Node **head, int value)
{
    Node *node = create_node(value);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
}
```

```
void display(Node *head)
{
    Node *current = head;

    printf("Linked list: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *head = NULL;

    insert_end(&head, 10);
    insert_end(&head, 20);
    insert_end(&head, 30);
    insert_end(&head, 40);

    display(head);

    free_list(head);

    return 0;
}
```

Output Text:

```
Linked list: 10 20 30 40
```

Program No: 42 | **Date:** 25-09-2026

Program Title: Insert a Node in a Linked List

Description:

Inserts nodes at the beginning, at the end, and after a specified element.

Author: Abhishek Vincent Roll No: 03 Date: 25-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->next = NULL;

    return node;
}

void insert_beginning(Node **head, int value)
{
    Node *node = create_node(value);

    if (node == NULL)
    {
        return;
    }

    node->next = *head;
    *head = node;
}

void insert_end(Node **head, int value)
{
    Node *node = create_node(value);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
```

```
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
}

void insert_after(Node *head, int target, int value)
{
    Node *current = head;
    Node *node;

    while (current != NULL && current->data != target)
    {
        current = current->next;
    }

    if (current == NULL)
    {
        printf("Element %d not found.\n", target);
        return;
    }

    node = create_node(value);

    if (node == NULL)
    {
        return;
    }

    node->next = current->next;
    current->next = node;
}

void display(Node *head)
{
    Node *current = head;

    printf("Linked list: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
```

```
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *head = NULL;

    insert_end(&head, 20);
    insert_end(&head, 30);
    insert_beginning(&head, 10);
    insert_after(head, 20, 25);
    insert_after(head, 30, 35);

    display(head);

    free_list(head);

    return 0;
}
```

Output Text:

```
Linked list: 10 20 25 30 35
```

Program No: 43 | **Date:** 25-09-2026

Program Title: Delete a Node from a Linked List

Description:

Deletes nodes from the beginning, from the end, and by value.

Author: Abhishek Vincent Roll No: 03 Date: 25-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->next = NULL;

    return node;
}

void insert_end(Node **head, int value)
{
    Node *node = create_node(value);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
}
```

```
void delete_beginning(Node **head)
{
    Node *temporary;

    if (*head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temporary = *head;
    *head = (*head)->next;
    free(temporary);
}

void delete_end(Node **head)
{
    Node *current;
    Node *previous;

    if (*head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if ((*head)->next == NULL)
    {
        free(*head);
        *head = NULL;
        return;
    }

    current = *head;
    previous = NULL;

    while (current->next != NULL)
    {
        previous = current;
        current = current->next;
    }

    previous->next = NULL;
    free(current);
}

void delete_value(Node **head, int value)
{
    Node *current = *head;
    Node *previous = NULL;

    while (current != NULL && current->data != value)
    {
        previous = current;
        current = current->next;
    }

    if (current == NULL)
    {
        printf("Element %d not found.\n", value);
        return;
    }
```

```
    if (previous == NULL)
    {
        *head = current->next;
    }
    else
    {
        previous->next = current->next;
    }

    free(current);
}

void display(Node *head)
{
    Node *current = head;

    printf("Linked list: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *head = NULL;

    insert_end(&head, 10);
    insert_end(&head, 20);
    insert_end(&head, 30);
    insert_end(&head, 40);

    delete_beginning(&head);
    delete_end(&head);
    delete_value(&head, 20);

    display(head);

    free_list(head);

    return 0;
}
```

Output Text:

```
Linked list: 30
```

Program No: 44 | **Date:** 25-09-2026

Program Title: Display a Linked List in Reverse Order

Description:

Creates a singly linked list of n nodes and displays it in reverse order using recursion.

Author: Abhishek Vincent Roll No: 03 Date: 25-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->next = NULL;

    return node;
}

void insert_end(Node **head, int value)
{
    Node *node = create_node(value);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
}
```

```
void display(Node *head)
{
    Node *current = head;

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void display_reverse(Node *head)
{
    if (head == NULL)
    {
        return;
    }

    display_reverse(head->next);
    printf("%d ", head->data);
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *head = NULL;
    int count;
    int value;
    int i;

    printf("Enter the number of nodes: ");
    scanf("%d", &count);

    if (count < 1)
    {
        printf("Invalid number of nodes.\n");
        return 0;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter value %d: ", i + 1);
        scanf("%d", &value);
        insert_end(&head, value);
    }

    printf("List in original order: ");
    display(head);
```

```
    printf("List in reverse order: ");
    display_reverse(head);
    printf("\n");

    free_list(head);

    return 0;
}
```

Output Text:

```
Enter the number of nodes: 5
Enter value 1: 10
Enter value 2: 20
Enter value 3: 30
Enter value 4: 40
Enter value 5: 50
List in original order: 10 20 30 40 50
List in reverse order: 50 40 30 20 10
```



Program No: 45 | **Date:** 25-09-2026

Program Title: Sort a Linked List

Description:

Sorts a linked list by swapping values and by swapping addresses.

Author: Abhishek Vincent Roll No: 03 Date: 25-09-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->next = NULL;

    return node;
}

void insert_end(Node **head, int value)
{
    Node *node = create_node(value);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
}
```

```
void display(Node *head)
{
    Node *current = head;

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void sort_by_value(Node *head)
{
    Node *current;
    Node *next;
    int temporary;
    int swapped;

    do
    {
        swapped = 0;
        current = head;

        while (current != NULL && current->next != NULL)
        {
            next = current->next;

            if (current->data > next->data)
            {
                temporary = current->data;
                current->data = next->data;
                next->data = temporary;
                swapped = 1;
            }

            current = current->next;
        }
    } while (swapped == 1);
}

void sort_by_address(Node **head)
{
    Node **current;
    Node *first;
    Node *second;
    int swapped;

    do
    {
        swapped = 0;
        current = head;

        while (*current != NULL && (*current)->next != NULL)
        {
            first = *current;
            second = first->next;

            if (first->data > second->data)
            {
                first->next = second->next;
```

```
                second->next = first;
                *current = second;
                swapped = 1;
            }

            current = &(*current)->next;
        }
    } while (swapped == 1);
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *first = NULL;
    Node *second = NULL;

    insert_end(&first, 30);
    insert_end(&first, 10);
    insert_end(&first, 40);
    insert_end(&first, 20);

    printf("Before sorting: ");
    display(first);

    sort_by_value(first);

    printf("Sorted by changing values: ");
    display(first);

    insert_end(&second, 30);
    insert_end(&second, 10);
    insert_end(&second, 40);
    insert_end(&second, 20);

    sort_by_address(&second);

    printf("Sorted by changing addresses: ");
    display(second);

    free_list(first);
    free_list(second);

    return 0;
}
```

Output Text:

```
Before sorting: 30 10 40 20
Sorted by changing values: 10 20 30 40
Sorted by changing addresses: 10 20 30 40
```

Program No: 46 | **Date:** 02-10-2026

Program Title: Polynomial Using a Linked List

Description:

Adds and multiplies two polynomials represented using linked lists.

Author: Abhishek Vincent Roll No: 03 Date: 02-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Term
{
    int coefficient;
    int exponent;
    struct Term *next;
} Term;

Term *create_term(int coefficient, int exponent)
{
    Term *term = malloc(sizeof(Term));

    if (term == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    term->coefficient = coefficient;
    term->exponent = exponent;
    term->next = NULL;

    return term;
}

void insert_term(Term **head, int coefficient, int exponent)
{
    Term **current = head;
    Term *term;

    while (*current != NULL && (*current)->exponent > exponent)
    {
        current = &(*current)->next;
    }

    if (*current != NULL && (*current)->exponent == exponent)
    {
        (*current)->coefficient += coefficient;

        if ((*current)->coefficient == 0)
        {
            term = *current;
            *current = (*current)->next;
            free(term);
        }

        return;
    }
```

```
    term = create_term(coefficient, exponent);

    if (term == NULL)
    {
        return;
    }

    term->next = *current;
    *current = term;
}

Term *read_polynomial(void)
{
    Term *head = NULL;
    int count;
    int coefficient;
    int exponent;
    int i;

    printf("Enter the number of terms: ");
    scanf("%d", &count);

    for (i = 0; i < count; i++)
    {
        printf("Term %d coefficient: ", i + 1);
        scanf("%d", &coefficient);
        printf("Term %d exponent: ", i + 1);
        scanf("%d", &exponent);
        insert_term(&head, coefficient, exponent);
    }

    return head;
}

void display_polynomial(Term *head)
{
    Term *current = head;
    int printed = 0;

    while (current != NULL)
    {
        int coefficient = current->coefficient;
        int exponent = current->exponent;

        if (coefficient != 0)
        {
            if (printed == 1 && coefficient > 0)
            {
                printf(" + ");
            }
            else if (printed == 1 && coefficient < 0)
            {
                printf(" - ");
            }
            else if (coefficient < 0)
            {
                printf("-");
            }

            if (coefficient < 0)
            {
                coefficient = -coefficient;
```

```
            }

            if (exponent == 0)
            {
                printf("%d", coefficient);
            }
            else if (exponent == 1)
            {
                if (coefficient != 1)
                {
                    printf("%d", coefficient);
                }
                printf("x");
            }
            else
            {
                if (coefficient != 1)
                {
                    printf("%d", coefficient);
                }
                printf("x^%d", exponent);
            }

            printed = 1;
        }

        current = current->next;
    }

    if (printed == 0)
    {
        printf("0");
    }

    printf("\n");
}

Term *add_polynomials(Term *first, Term *second)
{
    Term *result = NULL;
    Term *current;

    for (current = first; current != NULL; current = current->next)
    {
        insert_term(&result, current->coefficient, current->exponent);
    }

    for (current = second; current != NULL; current = current->next)
    {
        insert_term(&result, current->coefficient, current->exponent);
    }

    return result;
}

Term *multiply_polynomials(Term *first, Term *second)
{
    Term *result = NULL;
    Term *left;
    Term *right;

    for (left = first; left != NULL; left = left->next)
    {
```

```
        for (right = second; right != NULL; right = right->next)
        {
            insert_term(&result,
                        left->coefficient * right->coefficient,
                        left->exponent + right->exponent);
        }
    }

    return result;
}

void free_polynomial(Term *head)
{
    Term *current = head;
    Term *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Term *first;
    Term *second;
    Term *sum;
    Term *product;

    printf("First polynomial:\n");
    first = read_polynomial();

    printf("Second polynomial:\n");
    second = read_polynomial();

    printf("First polynomial: ");
    display_polynomial(first);

    printf("Second polynomial: ");
    display_polynomial(second);

    sum = add_polynomials(first, second);
    printf("Sum: ");
    display_polynomial(sum);

    product = multiply_polynomials(first, second);
    printf("Product: ");
    display_polynomial(product);

    free_polynomial(first);
    free_polynomial(second);
    free_polynomial(sum);
    free_polynomial(product);

    return 0;
}
```

Output Text:

```
First polynomial:
Enter the number of terms: 3
```

```
Term 1 coefficient: 2
Term 1 exponent: 2
Term 2 coefficient: 3
Term 2 exponent: 1
Term 3 coefficient: 1
Term 3 exponent: 0
Second polynomial:
Enter the number of terms: 2
Term 1 coefficient: 1
Term 1 exponent: 1
Term 2 coefficient: 4
Term 2 exponent: 0
First polynomial: 2x^2 + 3x + 1
Second polynomial: x + 4
Sum: 2x^2 + 4x + 5
Product: 2x^3 + 11x^2 + 13x + 4
```

Program No: 47 | **Date:** 02-10-2026

Program Title: Linked List of Names

Description:

Performs insertion, deletion, display, sorting, reversal, and counting on a linked list of names.

Author: Abhishek Vincent Roll No: 03 Date: 02-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_NAME 50

typedef struct Node
{
    char name[MAX_NAME];
    struct Node *next;
} Node;

Node *create_node(const char *name)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    strcpy(node->name, name);
    node->next = NULL;

    return node;
}

void insert_end(Node **head, const char *name)
{
    Node *node = create_node(name);
    Node *current;

    if (node == NULL)
    {
        return;
    }

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }
```

```
    current->next = node;
}

void delete_name(Node **head, const char *name)
{
    Node *current = *head;
    Node *previous = NULL;

    while (current != NULL && strcmp(current->name, name) != 0)
    {
        previous = current;
        current = current->next;
    }

    if (current == NULL)
    {
        printf("%s not found.\n", name);
        return;
    }

    if (previous == NULL)
    {
        *head = current->next;
    }
    else
    {
        previous->next = current->next;
    }

    free(current);
}

void display(Node *head)
{
    Node *current = head;

    printf("List: ");

    while (current != NULL)
    {
        printf("%s ", current->name);
        current = current->next;
    }

    printf("\n");
}

void sort(Node *head)
{
    Node *current;
    Node *next;
    char temporary[MAX_NAME];
    int swapped;

    do
    {
        swapped = 0;
        current = head;

        while (current != NULL && current->next != NULL)
        {
            next = current->next;
```

```
            if (strcmp(current->name, next->name) > 0)
            {
                strcpy(temporary, current->name);
                strcpy(current->name, next->name);
                strcpy(next->name, temporary);
                swapped = 1;
            }

            current = current->next;
        }
    } while (swapped == 1);
}

void reverse(Node **head)
{
    Node *previous = NULL;
    Node *current = *head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        current->next = previous;
        previous = current;
        current = next;
    }

    *head = previous;
}

int count_nodes(Node *head)
{
    int count = 0;
    Node *current = head;

    while (current != NULL)
    {
        count++;
        current = current->next;
    }

    return count;
}

void free_list(Node *head)
{
    Node *current = head;
    Node *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

int main(void)
{
    Node *head = NULL;

    insert_end(&head, "Alice");
```

```
    insert_end(&head, "Bob");
    insert_end(&head, "Carol");
    insert_end(&head, "Dave");

    display(head);
    printf("Count: %d\n", count_nodes(head));

    delete_name(&head, "Bob");
    display(head);

    sort(head);
    printf("After sorting: ");
    display(head);

    reverse(&head);
    printf("After reversing: ");
    display(head);

    free_list(head);

    return 0;
}
```

Output Text:

```
List: Alice Bob Carol Dave
Count: 4
List: Alice Carol Dave
After sorting: List: Alice Carol Dave
After reversing: List: Dave Carol Alice
```

Program No: 48 | **Date:** 09-10-2026

Program Title: Linked Stack, Queue, and Linked Lists

Description:

Demonstrates linked stack, linked queue, circular linked list, circular linked queue, doubly linked list, and circular doubly linked list.

Author: Abhishek Vincent Roll No: 03 Date: 09-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_NAME 50

typedef struct StackNode
{
    int data;
    struct StackNode *next;
} StackNode;

typedef struct QueueNode
{
    int data;
    struct QueueNode *next;
} QueueNode;

typedef struct CircularNode
{
    int data;
    struct CircularNode *next;
} CircularNode;

typedef struct CircularQueueNode
{
    int data;
    struct CircularQueueNode *next;
} CircularQueueNode;

typedef struct DoubleNode
{
    int data;
    struct DoubleNode *prev;
    struct DoubleNode *next;
} DoubleNode;

typedef struct CircularDoubleNode
{
    char name[MAX_NAME];
    struct CircularDoubleNode *prev;
    struct CircularDoubleNode *next;
} CircularDoubleNode;

void stack_push(StackNode **top, int value)
{
    StackNode *node = malloc(sizeof(StackNode));

    if (node == NULL)
```

```
    {
        printf("Memory allocation failed.\n");
        return;
    }

    node->data = value;
    node->next = *top;
    *top = node;
}

int stack_pop(StackNode **top)
{
    StackNode *temporary;
    int value;

    if (*top == NULL)
    {
        printf("Stack is empty.\n");
        return -1;
    }

    temporary = *top;
    value = temporary->data;
    *top = temporary->next;
    free(temporary);

    return value;
}

void stack_display(StackNode *top)
{
    StackNode *current = top;

    printf("Stack: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void stack_free(StackNode *top)
{
    StackNode *current = top;
    StackNode *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

void demo_linked_stack(void)
{
    StackNode *top = NULL;

    stack_push(&top, 10);
    stack_push(&top, 20);
```

```
    stack_push(&top, 30);

    stack_display(top);

    printf("Popped: %d\n", stack_pop(&top));
    stack_display(top);

    stack_free(top);
}

void queue_enqueue(QueueNode **front, QueueNode **rear, int value)
{
    QueueNode *node = malloc(sizeof(QueueNode));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    node->data = value;
    node->next = NULL;

    if (*rear == NULL)
    {
        *front = node;
        *rear = node;
    }
    else
    {
        (*rear)->next = node;
        *rear = node;
    }
}

int queue_dequeue(QueueNode **front, QueueNode **rear)
{
    QueueNode *temporary;
    int value;

    if (*front == NULL)
    {
        printf("Queue is empty.\n");
        return -1;
    }

    temporary = *front;
    value = temporary->data;
    *front = temporary->next;

    if (*front == NULL)
    {
        *rear = NULL;
    }

    free(temporary);

    return value;
}

void queue_display(QueueNode *front)
{
    QueueNode *current = front;
```

```
    printf("Queue: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void queue_free(QueueNode *front)
{
    QueueNode *current = front;
    QueueNode *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

void demo_linked_queue(void)
{
    QueueNode *front = NULL;
    QueueNode *rear = NULL;

    queue_enqueue(&front, &rear, 10);
    queue_enqueue(&front, &rear, 20);
    queue_enqueue(&front, &rear, 30);

    queue_display(front);

    printf("Dequeued: %d\n", queue_dequeue(&front, &rear));
    queue_display(front);

    queue_enqueue(&front, &rear, 40);
    queue_display(front);

    queue_free(front);
}

void circular_insert_end(CircularNode **head, int value)
{
    CircularNode *node = malloc(sizeof(CircularNode));
    CircularNode *current;

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    node->data = value;
    node->next = node;

    if (*head == NULL)
    {
        *head = node;
        return;
```

```
    }

    current = *head;

    while (current->next != *head)
    {
        current = current->next;
    }

    current->next = node;
    node->next = *head;
}

void circular_display(CircularNode *head)
{
    CircularNode *current;

    if (head == NULL)
    {
        printf("Circular list is empty.\n");
        return;
    }

    printf("Circular list: ");
    current = head;

    do
    {
        printf("%d ", current->data);
        current = current->next;
    } while (current != head);

    printf("\n");
}

void circular_delete(CircularNode **head, int value)
{
    CircularNode *current = *head;
    CircularNode *previous = NULL;

    if (*head == NULL)
    {
        printf("Circular list is empty.\n");
        return;
    }

    do
    {
        if (current->data == value)
        {
            break;
        }

        previous = current;
        current = current->next;
    } while (current != *head);

    if (current->data != value)
    {
        printf("%d not found.\n", value);
        return;
    }
```

```
    if (current->next == current)
    {
        *head = NULL;
    }
    else
    {
        if (previous == NULL)
        {
            previous = *head;

            while (previous->next != *head)
            {
                previous = previous->next;
            }

            *head = current->next;
        }

        previous->next = current->next;
    }

    free(current);
}

void circular_free(CircularNode *head)
{
    CircularNode *current;
    CircularNode *next;

    if (head == NULL)
    {
        return;
    }

    current = head;

    do
    {
        next = current->next;
        free(current);
        current = next;
    } while (current != head);
}

void demo_circular_linked_list(void)
{
    CircularNode *head = NULL;

    circular_insert_end(&head, 10);
    circular_insert_end(&head, 20);
    circular_insert_end(&head, 30);

    circular_display(head);

    circular_delete(&head, 20);
    circular_display(head);

    circular_free(head);
}

void circular_queue_enqueue(CircularQueueNode **rear, int value)
{
    CircularQueueNode *node = malloc(sizeof(CircularQueueNode));
```

```
    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    node->data = value;
    node->next = node;

    if (*rear == NULL)
    {
        *rear = node;
        return;
    }

    node->next = (*rear)->next;
    (*rear)->next = node;
    *rear = node;
}

int circular_queue_dequeue(CircularQueueNode **rear)
{
    CircularQueueNode *front;
    int value;

    if (*rear == NULL)
    {
        printf("Circular queue is empty.\n");
        return -1;
    }

    front = (*rear)->next;
    value = front->data;

    if (front == *rear)
    {
        *rear = NULL;
    }
    else
    {
        (*rear)->next = front->next;
    }

    free(front);

    return value;
}

void circular_queue_display(CircularQueueNode *rear)
{
    CircularQueueNode *front;
    CircularQueueNode *current;

    if (rear == NULL)
    {
        printf("Circular queue is empty.\n");
        return;
    }

    printf("Circular queue: ");
    front = rear->next;
    current = front;
```

```
    do
    {
        printf("%d ", current->data);
        current = current->next;
    } while (current != front);

    printf("\n");
}

void demo_circular_linked_queue(void)
{
    CircularQueueNode *rear = NULL;

    circular_queue_enqueue(&rear, 10);
    circular_queue_enqueue(&rear, 20);
    circular_queue_enqueue(&rear, 30);

    circular_queue_display(rear);

    printf("Dequeued: %d\n", circular_queue_dequeue(&rear));
    circular_queue_display(rear);

    circular_queue_enqueue(&rear, 40);
    circular_queue_display(rear);

    while (rear != NULL)
    {
        circular_queue_dequeue(&rear);
    }
}

void double_insert_end(DoubleNode **head, int value)
{
    DoubleNode *node = malloc(sizeof(DoubleNode));
    DoubleNode *current;

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    node->data = value;
    node->prev = NULL;
    node->next = NULL;

    if (*head == NULL)
    {
        *head = node;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = node;
    node->prev = current;
}
```

```
void double_display_forward(DoubleNode *head)
{
    DoubleNode *current = head;

    printf("Forward: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    printf("\n");
}

void double_display_backward(DoubleNode *head)
{
    DoubleNode *current = head;

    if (current == NULL)
    {
        printf("Backward: empty\n");
        return;
    }

    while (current->next != NULL)
    {
        current = current->next;
    }

    printf("Backward: ");

    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->prev;
    }

    printf("\n");
}

void double_delete(DoubleNode **head, int value)
{
    DoubleNode *current = *head;

    while (current != NULL && current->data != value)
    {
        current = current->next;
    }

    if (current == NULL)
    {
        printf("%d not found.\n", value);
        return;
    }

    if (current->prev != NULL)
    {
        current->prev->next = current->next;
    }
    else
    {
```

```
        *head = current->next;
    }

    if (current->next != NULL)
    {
        current->next->prev = current->prev;
    }

    free(current);
}

void double_free(DoubleNode *head)
{
    DoubleNode *current = head;
    DoubleNode *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

void demo_doubly_linked_list(void)
{
    DoubleNode *head = NULL;

    double_insert_end(&head, 10);
    double_insert_end(&head, 20);
    double_insert_end(&head, 30);

    double_display_forward(head);
    double_display_backward(head);

    double_delete(&head, 20);

    double_display_forward(head);
    double_display_backward(head);

    double_free(head);
}

void circular_double_insert_end(CircularDoubleNode **head, const char *name)
{
    CircularDoubleNode *node = malloc(sizeof(CircularDoubleNode));
    CircularDoubleNode *tail;

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(node->name, name);

    if (*head == NULL)
    {
        node->next = node;
        node->prev = node;
        *head = node;
        return;
    }
```

```
    tail = (*head)->prev;
    node->next = *head;
    node->prev = tail;
    tail->next = node;
    (*head)->prev = node;
}

void circular_double_display_forward(CircularDoubleNode *head)
{
    CircularDoubleNode *current;

    if (head == NULL)
    {
        printf("Circular double list is empty.\n");
        return;
    }

    printf("Forward: ");
    current = head;

    do
    {
        printf("%s ", current->name);
        current = current->next;
    } while (current != head);

    printf("\n");
}

void circular_double_display_backward(CircularDoubleNode *head)
{
    CircularDoubleNode *tail;
    CircularDoubleNode *current;

    if (head == NULL)
    {
        printf("Circular double list is empty.\n");
        return;
    }

    printf("Backward: ");
    tail = head->prev;
    current = tail;

    do
    {
        printf("%s ", current->name);
        current = current->prev;
    } while (current != tail);

    printf("\n");
}

void circular_double_delete(CircularDoubleNode **head, const char *name)
{
    CircularDoubleNode *current = *head;

    if (*head == NULL)
    {
        printf("Circular double list is empty.\n");
        return;
    }
```

```
    do
    {
        if (strcmp(current->name, name) == 0)
        {
            break;
        }

        current = current->next;
    } while (current != *head);

    if (strcmp(current->name, name) != 0)
    {
        printf("%s not found.\n", name);
        return;
    }

    if (current->next == current)
    {
        *head = NULL;
    }
    else
    {
        current->prev->next = current->next;
        current->next->prev = current->prev;

        if (current == *head)
        {
            *head = current->next;
        }
    }

    free(current);
}

void circular_double_free(CircularDoubleNode *head)
{
    CircularDoubleNode *current;
    CircularDoubleNode *next;

    if (head == NULL)
    {
        return;
    }

    current = head;

    do
    {
        next = current->next;
        free(current);
        current = next;
    } while (current != head);
}

void demo_circular_doubly_linked_list(void)
{
    CircularDoubleNode *head = NULL;

    circular_double_insert_end(&head, "Alice");
    circular_double_insert_end(&head, "Bob");
    circular_double_insert_end(&head, "Carol");
```

```
    circular_double_display_forward(head);
    circular_double_display_backward(head);

    circular_double_delete(&head, "Bob");

    circular_double_display_forward(head);
    circular_double_display_backward(head);

    circular_double_free(head);
}

int main(void)
{
    printf("Linked Stack\n");
    demo_linked_stack();

    printf("\nLinked Queue\n");
    demo_linked_queue();

    printf("\nCircular Linked List\n");
    demo_circular_linked_list();

    printf("\nCircular Linked Queue\n");
    demo_circular_linked_queue();

    printf("\nDoubly Linked List\n");
    demo_doubly_linked_list();

    printf("\nCircular Doubly Linked List\n");
    demo_circular_doubly_linked_list();

    return 0;
}
```

Output Text:

```
Linked Stack
Stack: 30 20 10
Popped: 30
Stack: 20 10

Linked Queue
Queue: 10 20 30
Dequeued: 10
Queue: 20 30
Queue: 20 30 40

Circular Linked List
Circular list: 10 20 30
Circular list: 10 30

Circular Linked Queue
Circular queue: 10 20 30
Dequeued: 10
Circular queue: 20 30
Circular queue: 20 30 40

Doubly Linked List
Forward: 10 20 30
Backward: 30 20 10
Forward: 10 30
Backward: 30 10
```

```
Circular Doubly Linked List
Forward: Alice Bob Carol
Backward: Carol Bob Alice
Forward: Alice Carol
Backward: Carol Alice
```

Program No: 49 | **Date:** 16-10-2026

Program Title: Linear Search

Description:

Searches for an element in an array using linear search.

Author: Abhishek Vincent Roll No: 03 Date: 16-10-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

int linear_search(int array[], int size, int value)
{
    int i;

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            return i;
        }
    }

    return -1;
}

int main(void)
{
    int array[MAX];
    int size;
    int value;
    int position;
    int i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
    }

    for (i = 0; i < size; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &value);

    position = linear_search(array, size, value);

    if (position == -1)
    {
        printf("Element %d not found.\n", value);
```

```
    }
    else
    {
        printf("Element %d found at position %d.\n", value, position + 1);
    }

    return 0;
}
```

Output Text:

```
Enter the size of the array: 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Enter the element to search: 30
Element 30 found at position 3.
```

Program No: 50 | **Date:** 16-10-2026

Program Title: Binary Search for Integers

Description:

Searches for an element in a sorted integer array using binary search.

Author: Abhishek Vincent Roll No: 03 Date: 16-10-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

int binary_search(int array[], int size, int value)
{
    int low = 0;
    int high = size - 1;
    int middle;

    while (low <= high)
    {
        middle = (low + high) / 2;

        if (array[middle] == value)
        {
            return middle;
        }

        if (array[middle] < value)
        {
            low = middle + 1;
        }
        else
        {
            high = middle - 1;
        }
    }

    return -1;
}

int main(void)
{
    int array[MAX];
    int size;
    int value;
    int position;
    int i;

    printf("Enter the size of the sorted array: ");
    scanf("%d", &size);

    if (size < 1 || size > MAX)
    {
        printf("Invalid size.\n");
        return 0;
    }

    for (i = 0; i < size; i++)
```

```
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &array[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &value);

    position = binary_search(array, size, value);

    if (position == -1)
    {
        printf("Element %d not found.\n", value);
    }
    else
    {
        printf("Element %d found at position %d.\n", value, position + 1);
    }

    return 0;
}
```

Output Text:

```
Enter the size of the sorted array: 6
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Enter element 6: 60
Enter the element to search: 40
Element 40 found at position 4.
```

Program No: 51 | **Date:** 16-10-2026

Program Title: Binary Search for Strings

Description:

Searches for a string in a sorted array of strings using binary search.

Author: Abhishek Vincent Roll No: 03 Date: 16-10-2026

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_STRINGS 50
#define MAX_LENGTH 50

int binary_search_strings(char strings[][MAX_LENGTH], int count, char value[])
{
    int low = 0;
    int high = count - 1;
    int middle;
    int comparison;

    while (low <= high)
    {
        middle = (low + high) / 2;
        comparison = strcmp(strings[middle], value);

        if (comparison == 0)
        {
            return middle;
        }

        if (comparison < 0)
        {
            low = middle + 1;
        }
        else
        {
            high = middle - 1;
        }
    }

    return -1;
}

int main(void)
{
    char strings[MAX_STRINGS][MAX_LENGTH];
    char value[MAX_LENGTH];
    int count;
    int position;
    int i;

    printf("Enter the number of sorted strings: ");
    scanf("%d", &count);

    if (count < 1 || count > MAX_STRINGS)
    {
        printf("Invalid number of strings.\n");
```

```
        return 0;
    }

    for (i = 0; i < count; i++)
    {
        printf("Enter string %d: ", i + 1);
        scanf(" %49[^\n]", strings[i]);
    }

    printf("Enter the string to search: ");
    scanf(" %49[^\n]", value);

    position = binary_search_strings(strings, count, value);

    if (position == -1)
    {
        printf("%s not found.\n", value);
    }
    else
    {
        printf("%s found at position %d.\n", value, position + 1);
    }

    return 0;
}
```

Output Text:

```
Enter the number of sorted strings: 5
Enter string 1: apple
Enter string 2: banana
Enter string 3: cherry
Enter string 4: date
Enter string 5: elder
Enter the string to search: cherry
cherry found at position 3.
```

Program No: 52 | **Date:** 23-10-2026

Program Title: Binary Search Tree Traversals

Description:

Inserts values into a binary search tree and displays inorder, preorder, and postorder traversals using recursion.

Author: Abhishek Vincent Roll No: 03 Date: 23-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
    {
        return create_node(value);
    }

    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else if (value > root->data)
    {
        root->right = insert(root->right, value);
    }

    return root;
}

void inorder(Node *root)
{
    if (root == NULL)
```

```
    {
        return;
    }

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void preorder(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

void free_tree(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void)
{
    Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Inorder: ");
    inorder(root);
    printf("\n");

    printf("Preorder: ");
    preorder(root);
```

```
    printf("\n");

    printf("Postorder: ");
    postorder(root);
    printf("\n");

    free_tree(root);

    return 0;
}
```

Output Text:

```
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
```

Program No: 53 | **Date:** 23-10-2026

Program Title: Inorder Traversal Without Recursion

Description:

Displays the inorder traversal of a binary search tree without recursion.

Author: Abhishek Vincent Roll No: 03 Date: 23-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
    {
        return create_node(value);
    }

    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else if (value > root->data)
    {
        root->right = insert(root->right, value);
    }

    return root;
}

void inorder_iterative(Node *root)
{
```

```
    Node *stack[MAX];
    int top = -1;
    Node *current = root;

    while (current != NULL || top != -1)
    {
        while (current != NULL)
        {
            top++;
            stack[top] = current;
            current = current->left;
        }

        current = stack[top];
        top--;
        printf("%d ", current->data);
        current = current->right;
    }
}

void free_tree(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void)
{
    Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Inorder without recursion: ");
    inorder_iterative(root);
    printf("\n");

    free_tree(root);

    return 0;
}
```

Output Text:

```
Inorder without recursion: 20 30 40 50 60 70 80
```

Program No: 54 | **Date:** 23-10-2026

Program Title: Preorder Traversal Without Recursion

Description:

Displays the preorder traversal of a binary search tree without recursion.

Author: Abhishek Vincent Roll No: 03 Date: 23-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
    {
        return create_node(value);
    }

    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else if (value > root->data)
    {
        root->right = insert(root->right, value);
    }

    return root;
}

void preorder_iterative(Node *root)
{
```

```
    Node *stack[MAX];
    int top = -1;
    Node *current;

    if (root != NULL)
    {
        top++;
        stack[top] = root;
    }

    while (top != -1)
    {
        current = stack[top];
        top--;

        printf("%d ", current->data);

        if (current->right != NULL)
        {
            top++;
            stack[top] = current->right;
        }

        if (current->left != NULL)
        {
            top++;
            stack[top] = current->left;
        }
    }
}

void free_tree(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void)
{
    Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Preorder without recursion: ");
    preorder_iterative(root);
    printf("\n");

    free_tree(root);

    return 0;
}
```

Output Text:

```
Preorder without recursion: 50 30 20 40 70 60 80
```

Program No: 55 | **Date:** 23-10-2026

Program Title: Postorder Traversal Without Recursion

Description:

Displays the postorder traversal of a binary search tree without recursion.

Author: Abhishek Vincent Roll No: 03 Date: 23-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node *create_node(int value)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    node->data = value;
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *insert(Node *root, int value)
{
    if (root == NULL)
    {
        return create_node(value);
    }

    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else if (value > root->data)
    {
        root->right = insert(root->right, value);
    }

    return root;
}

void postorder_iterative(Node *root)
{
```

```
    Node *first[MAX];
    Node *second[MAX];
    int first_top = -1;
    int second_top = -1;
    Node *current;

    if (root != NULL)
    {
        first_top++;
        first[first_top] = root;
    }

    while (first_top != -1)
    {
        current = first[first_top];
        first_top--;

        second_top++;
        second[second_top] = current;

        if (current->left != NULL)
        {
            first_top++;
            first[first_top] = current->left;
        }

        if (current->right != NULL)
        {
            first_top++;
            first[first_top] = current->right;
        }
    }

    while (second_top != -1)
    {
        printf("%d ", second[second_top]->data);
        second_top--;
    }
}

void free_tree(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void)
{
    Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);
```

```
    printf("Postorder without recursion: ");
    postorder_iterative(root);
    printf("\n");

    free_tree(root);

    return 0;
}
```

Output Text:

```
Postorder without recursion: 20 40 30 60 80 70 50
```

Program No: 56 | **Date:** 23-10-2026

Program Title: Binary Search Tree Using Names

Description:

Inserts names into a binary search tree and displays them in ascending order using inorder traversal.

Author: Abhishek Vincent Roll No: 03 Date: 23-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_NAME 50

typedef struct Node
{
    char name[MAX_NAME];
    struct Node *left;
    struct Node *right;
} Node;

Node *create_node(const char *name)
{
    Node *node = malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    strcpy(node->name, name);
    node->left = NULL;
    node->right = NULL;

    return node;
}

Node *insert(Node *root, const char *name)
{
    if (root == NULL)
    {
        return create_node(name);
    }

    if (strcmp(name, root->name) < 0)
    {
        root->left = insert(root->left, name);
    }
    else if (strcmp(name, root->name) > 0)
    {
        root->right = insert(root->right, name);
    }

    return root;
}

void inorder(Node *root)
```

```
{
    if (root == NULL)
    {
        return;
    }

    inorder(root->left);
    printf("%s ", root->name);
    inorder(root->right);
}

void free_tree(Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);
    free(root);
}

int main(void)
{
    Node *root = NULL;

    root = insert(root, "Ravi");
    root = insert(root, "Anu");
    root = insert(root, "Kiran");
    root = insert(root, "Bala");
    root = insert(root, "Deepa");
    root = insert(root, "Cathy");

    printf("Names in ascending order: ");
    inorder(root);
    printf("\n");

    free_tree(root);

    return 0;
}
```

Output Text:

```
Names in ascending order: Anu Bala Cathy Deepa Kiran Ravi
```

Program No: Project | **Date:** 30-10-2026

Program Title: Student Record Management System

Description:

Menu-driven student record management using a singly linked list with add, display, save, load, update, delete, and sort operations.

Author: Abhishek Vincent Roll No: 03 Date: 30-10-2026

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NAME_LENGTH 50
#define SUBJECTS 5
#define FILE_NAME "students.txt"

typedef struct Student
{
    int roll;
    char name[NAME_LENGTH];
    int marks[SUBJECTS];
    int total;
    struct Student *next;
} Student;

Student *create_student(int roll, const char *name, int marks[])
{
    Student *student = malloc(sizeof(Student));
    int i;

    if (student == NULL)
    {
        printf("Memory allocation failed.\n");
        return NULL;
    }

    student->roll = roll;
    strcpy(student->name, name);
    student->total = 0;

    for (i = 0; i < SUBJECTS; i++)
    {
        student->marks[i] = marks[i];
        student->total += marks[i];
    }

    student->next = NULL;

    return student;
}

void insert_end(Student **head, Student *student)
{
    Student *current;

    if (student == NULL)
    {
```

```
        return;
    }

    if (*head == NULL)
    {
        *head = student;
        return;
    }

    current = *head;

    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = student;
}

Student *find_student(Student *head, int roll)
{
    Student *current = head;

    while (current != NULL)
    {
        if (current->roll == roll)
        {
            return current;
        }

        current = current->next;
    }

    return NULL;
}

void read_marks(int marks[])
{
    int i;

    for (i = 0; i < SUBJECTS; i++)
    {
        printf("Enter mark %d: ", i + 1);
        scanf("%d", &marks[i]);
    }
}

void add_student(Student **head)
{
    int roll;
    char name[NAME_LENGTH];
    int marks[SUBJECTS];
    Student *student;

    printf("Enter roll number: ");
    scanf("%d", &roll);

    if (find_student(*head, roll) != NULL)
    {
        printf("Roll number %d already exists.\n", roll);
        return;
    }
```

```
    printf("Enter name: ");
    scanf(" %49[^\n]", name);
    read_marks(marks);

    student = create_student(roll, name, marks);

    if (student != NULL)
    {
        insert_end(head, student);
        printf("Student added.\n");
    }
}

void display_students(Student *head)
{
    Student *current = head;
    int i;

    if (current == NULL)
    {
        printf("No student records.\n");
        return;
    }

    printf("%-8s %-20s %-20s %-8s\n", "Roll", "Name", "Marks", "Total");

    while (current != NULL)
    {
        printf("%-8d %-20s ", current->roll, current->name);

        for (i = 0; i < SUBJECTS; i++)
        {
            printf("%d ", current->marks[i]);
        }

        printf("%-8d\n", current->total);
        current = current->next;
    }
}

void update_student(Student *head)
{
    int roll;
    char name[NAME_LENGTH];
    int marks[SUBJECTS];
    int i;
    Student *student;

    printf("Enter the roll number to update: ");
    scanf("%d", &roll);

    student = find_student(head, roll);

    if (student == NULL)
    {
        printf("Student %d not found.\n", roll);
        return;
    }

    printf("Enter the new name: ");
    scanf(" %49[^\n]", name);
    read_marks(marks);
```

```
    strcpy(student->name, name);
    student->total = 0;

    for (i = 0; i < SUBJECTS; i++)
    {
        student->marks[i] = marks[i];
        student->total += marks[i];
    }

    printf("Student updated.\n");
}

void delete_student(Student **head)
{
    int roll;
    Student *current = *head;
    Student *previous = NULL;

    printf("Enter the roll number to delete: ");
    scanf("%d", &roll);

    while (current != NULL && current->roll != roll)
    {
        previous = current;
        current = current->next;
    }

    if (current == NULL)
    {
        printf("Student %d not found.\n", roll);
        return;
    }

    if (previous == NULL)
    {
        *head = current->next;
    }
    else
    {
        previous->next = current->next;
    }

    free(current);
    printf("Student %d deleted.\n", roll);
}

void swap_data(Student *first, Student *second)
{
    int roll = first->roll;
    char name[NAME_LENGTH];
    int marks[SUBJECTS];
    int total = first->total;
    int i;

    strcpy(name, first->name);

    for (i = 0; i < SUBJECTS; i++)
    {
        marks[i] = first->marks[i];
    }

    first->roll = second->roll;
    strcpy(first->name, second->name);
```

```
    first->total = second->total;

    for (i = 0; i < SUBJECTS; i++)
    {
        first->marks[i] = second->marks[i];
    }

    second->roll = roll;
    strcpy(second->name, name);
    second->total = total;

    for (i = 0; i < SUBJECTS; i++)
    {
        second->marks[i] = marks[i];
    }
}

void sort_by_roll(Student *head)
{
    Student *current;
    Student *next;
    int swapped;

    do
    {
        swapped = 0;
        current = head;

        while (current != NULL && current->next != NULL)
        {
            next = current->next;

            if (current->roll > next->roll)
            {
                swap_data(current, next);
                swapped = 1;
            }

            current = current->next;
        }
    } while (swapped == 1);
}

void sort_by_name(Student *head)
{
    Student *current;
    Student *next;
    int swapped;

    do
    {
        swapped = 0;
        current = head;

        while (current != NULL && current->next != NULL)
        {
            next = current->next;

            if (strcmp(current->name, next->name) > 0)
            {
                swap_data(current, next);
                swapped = 1;
            }
```

```
            current = current->next;
        }
    } while (swapped == 1);
}

void save_to_file(Student *head)
{
    FILE *file = fopen(FILE_NAME, "w");
    Student *current = head;
    int i;

    if (file == NULL)
    {
        printf("Unable to open %s for writing.\n", FILE_NAME);
        return;
    }

    while (current != NULL)
    {
        fprintf(file, "%d|%s|", current->roll, current->name);

        for (i = 0; i < SUBJECTS; i++)
        {
            fprintf(file, "%d", current->marks[i]);

            if (i < SUBJECTS - 1)
            {
                fprintf(file, " ");
            }
        }

        fprintf(file, "\n");
        current = current->next;
    }

    fclose(file);
    printf("Records saved to %s.\n", FILE_NAME);
}

void free_list(Student *head)
{
    Student *current = head;
    Student *next;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }
}

void load_from_file(Student **head)
{
    FILE *file = fopen(FILE_NAME, "r");
    int roll;
    char name[NAME_LENGTH];
    int marks[SUBJECTS];
    Student *student;

    if (file == NULL)
    {
```

```
        printf("Unable to open %s for reading.\n", FILE_NAME);
        return;
    }

    free_list(*head);
    *head = NULL;

    while (fscanf(file, "%d|%49[^|]|%d %d %d %d %d",
                  &roll,
                  name,
                  &marks[0],
                  &marks[1],
                  &marks[2],
                  &marks[3],
                  &marks[4]) == 7)
    {
        student = create_student(roll, name, marks);

        if (student != NULL)
        {
            insert_end(head, student);
        }
    }

    fclose(file);
    printf("Records loaded from %s.\n", FILE_NAME);
}

void print_menu(void)
{
    printf("\n1. Add student\n");
    printf("2. Display students\n");
    printf("3. Save to file\n");
    printf("4. Load from file\n");
    printf("5. Update student\n");
    printf("6. Delete student\n");
    printf("7. Sort by roll number\n");
    printf("8. Sort by name\n");
    printf("9. Exit\n");
}

int main(void)
{
    Student *head = NULL;
    int choice;

    do
    {
        print_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                add_student(&head);
                break;
            case 2:
                display_students(head);
                break;
            case 3:
                save_to_file(head);
                break;
```

```
            case 4:
                load_from_file(&head);
                break;
            case 5:
                update_student(head);
                break;
            case 6:
                delete_student(&head);
                break;
            case 7:
                sort_by_roll(head);
                printf("Sorted by roll number.\n");
                break;
            case 8:
                sort_by_name(head);
                printf("Sorted by name.\n");
                break;
            case 9:
                printf("Program ended.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 9);

    free_list(head);

    return 0;
}
```

Output Text:

```
1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 1
Enter roll number: 101
Enter name: Alice
Enter mark 1: 80
Enter mark 2: 85
Enter mark 3: 90
Enter mark 4: 75
Enter mark 5: 88
Student added.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 1
Enter roll number: 102
```

```
Enter name: Bob
Enter mark 1: 70
Enter mark 2: 72
Enter mark 3: 68
Enter mark 4: 75
Enter mark 5: 80
Student added.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 1
Enter roll number: 103
Enter name: Carol
Enter mark 1: 90
Enter mark 2: 92
Enter mark 3: 88
Enter mark 4: 95
Enter mark 5: 91
Student added.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 2
Roll     Name               Marks          Total
101      Alice              80 85 90 75 88 418
102      Bob                70 72 68 75 80 365
103      Carol              90 92 88 95 91 456

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 5
Enter the roll number to update: 101
Enter the new name: Alice Johnson
Enter mark 1: 85
Enter mark 2: 86
Enter mark 3: 87
Enter mark 4: 88
Enter mark 5: 89
Student updated.

1. Add student
2. Display students
```

```
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 6
Enter the roll number to delete: 102
Student 102 deleted.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 7
Sorted by roll number.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 8
Sorted by name.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 2
Roll     Name                Marks                Total
101      Alice Johnson       85 86 87 88 89 435
103      Carol               90 92 88 95 91 456

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 3
Records saved to students.txt.

1. Add student
2. Display students
```

```
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 4
Records loaded from students.txt.

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 2
Roll     Name                Marks                Total
101      Alice Johnson       85 86 87 88 89 435
103      Carol               90 92 88 95 91 456

1. Add student
2. Display students
3. Save to file
4. Load from file
5. Update student
6. Delete student
7. Sort by roll number
8. Sort by name
9. Exit
Enter your choice: 9
Program ended.
```